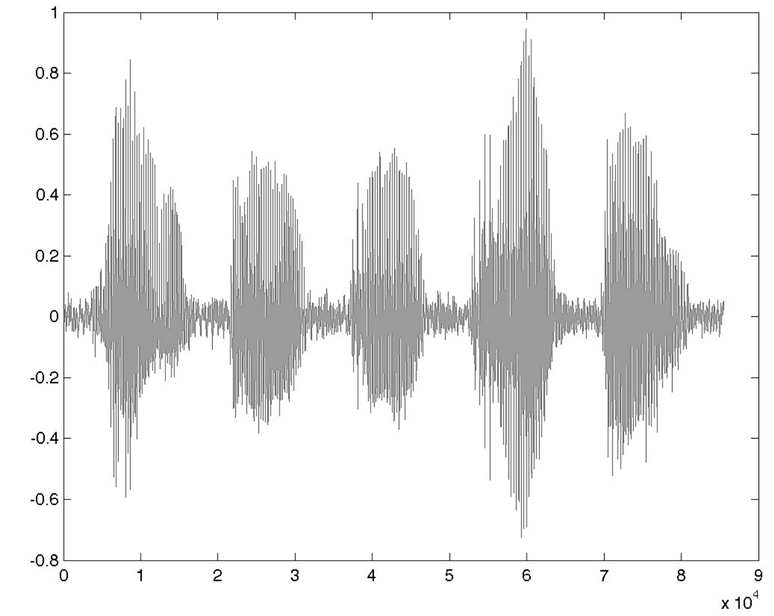
Audio Computing Lab

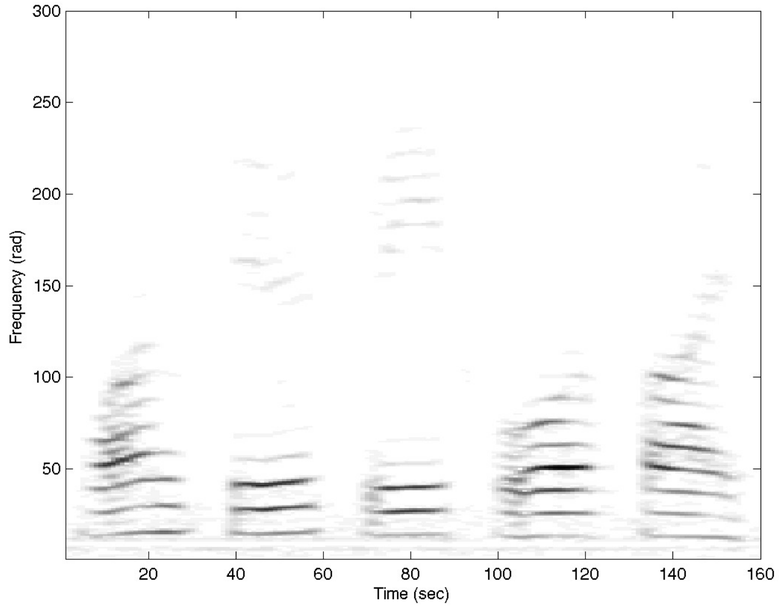
Audio Signal Processing basics

Paris Smaragdis
paris@illinois.edu
paris.cs.illinois.edu

Overview

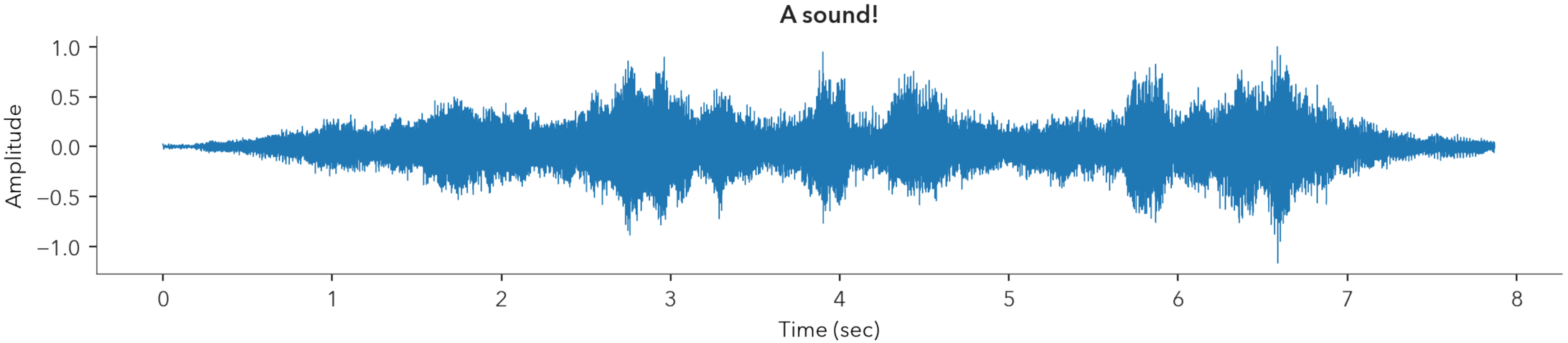
- Basics of digital audio
- Signal representations
  - Time, Frequency, Time/Frequency
  - Sampling, Quantization
- The Fourier transform
  - DFT and FFT
- The Spectrogram





Sound as “numbers”

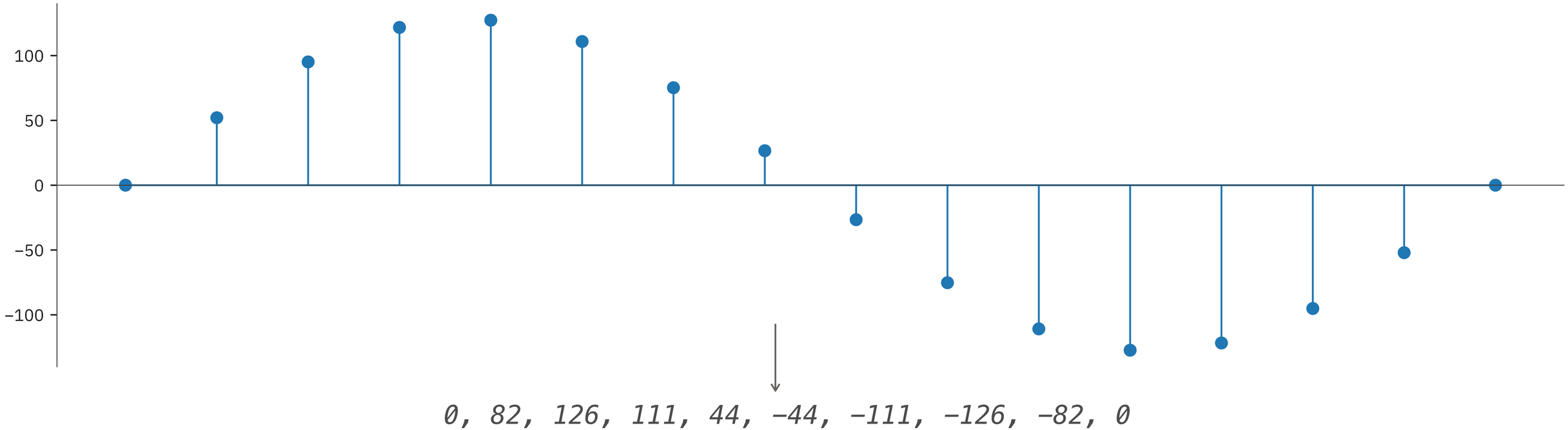
- We treat sound as a series of amplitudes
  - More on the details later



- This is the *waveform* representation
  - Encodes instantaneous pressure over time

PCM format

- "Pulse Code Modulation"
  - Used by CDs, telephones, audio editors, synths, etc.



This is a *discrete* and *digital* format

- We can not use continuous waveforms
  - Discrete: We have finite samples over time
  - Digital: We (usually) encode these samples as signed integers
- Common formats
  - Speech: 16 kHz / 16-bit (or 8-bit)
  - Music: 44.1kHz/ 16-bit (or 96kHz / 24-bit)
- These numbers characterize the *sampling* process
  - But why do we use these values?

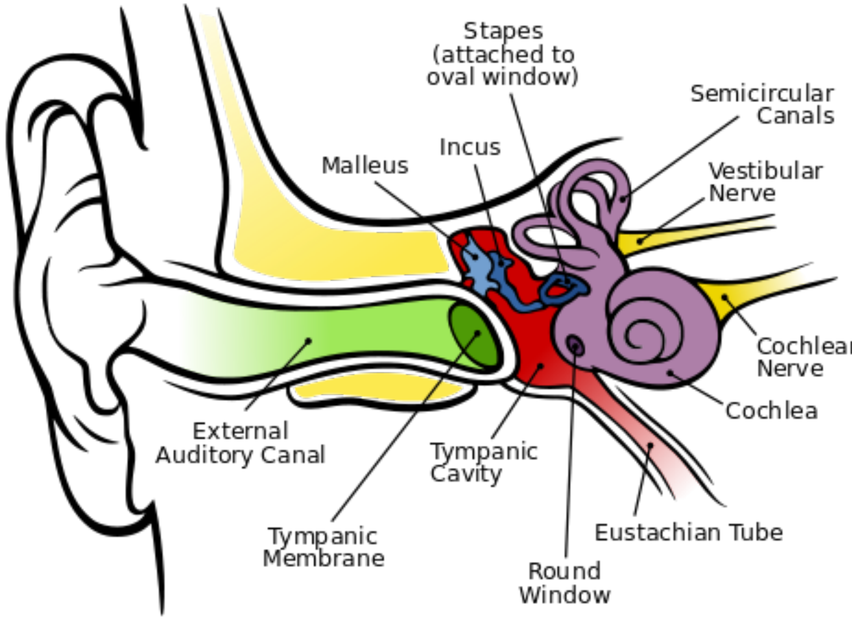
Dynamic range

- The choice of bits defines the *dynamic range*
  - More bits == more dynamic range == more storage needs
- What is dynamic range?
  - Ratio of highest and lowest represented pressure value
  - Usually measured in *decibels* (dB)
- So how much dynamic range do we need?

It all hinges on how we hear

- Outer ear
  - Sound gets collected at the *pinna*
  - The *ear canal* amplifies (some) sound by ~10dB
  - The *ear drum* vibrates according to incoming pressure

- Middle ear
  - The *ossicles* transfer sound to the *oval window*
  - Amplify sound by ~14dB
    - Also use muscles for damping



- Inner ear
  - Translation to neural signal (more later)

Perception of loudness

- The *just noticeable* sound is:
  - 10^{-12} W/m^2 (we cannot hear softer than this)
- And the as noticeable as it get is:
  - 1 W/m^2 (more than that and you go deaf!)
- Thus our dynamic range is:
  - $10 \log_{10}(1/10^{-12}) = 120$ dB
  - That's a staggering trillion to one!

To get you oriented

• Weakest detectable sound	~0 dB
• Soft breathing	~10dB
• Quiet library	~40 dB
• Office environment	~60 dB
• Food blender	~80 dB
• Lawn mower	~90 dB
• Car horn at 1m	~110 dB
• Military jet at 15m	~130 dB
• Shotgun blast	~165 dB
• Loudest possible sound	194 dB

Dangerous levels > 90 dB

Pain begins at 125 dB

Pain ends at 180 dB

(cause your ears just blew up)

  - (after which it is not a "sound" anymore it is called a *shock wave*)

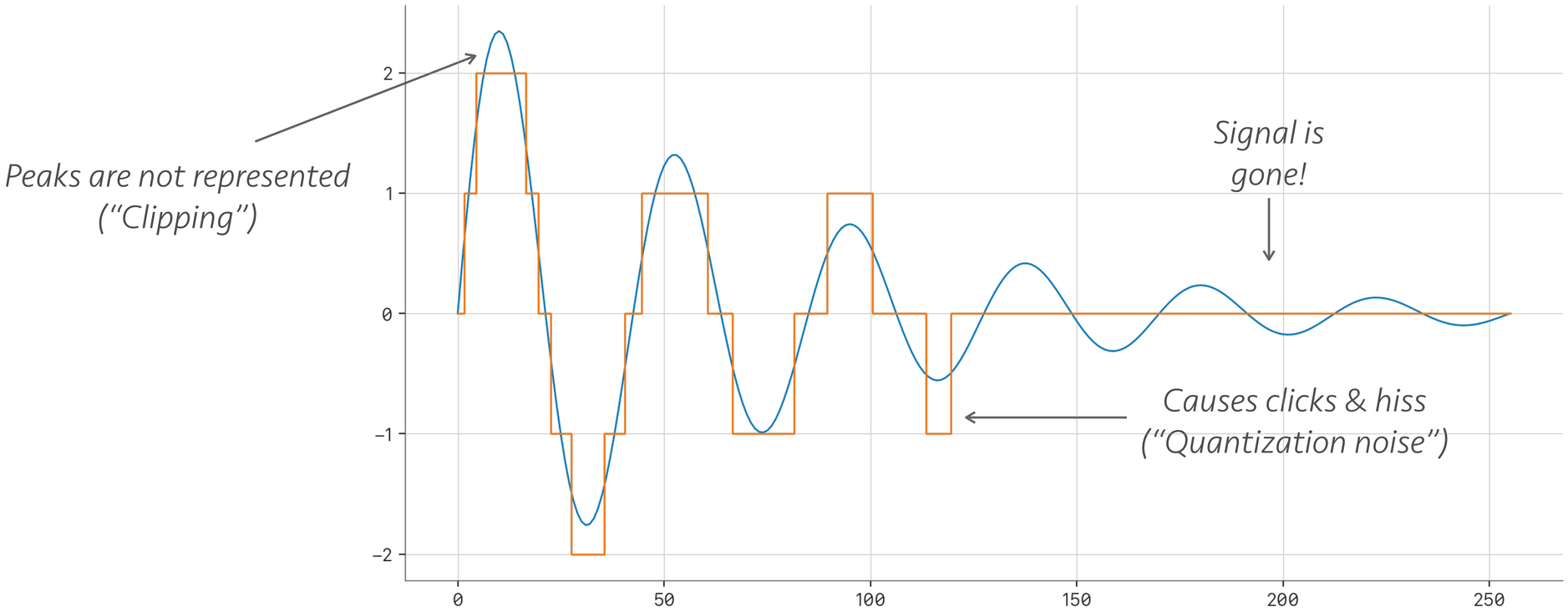
Back to digital sound

- How many dB dynamic range to use?
  - Close to 120 dB ideally

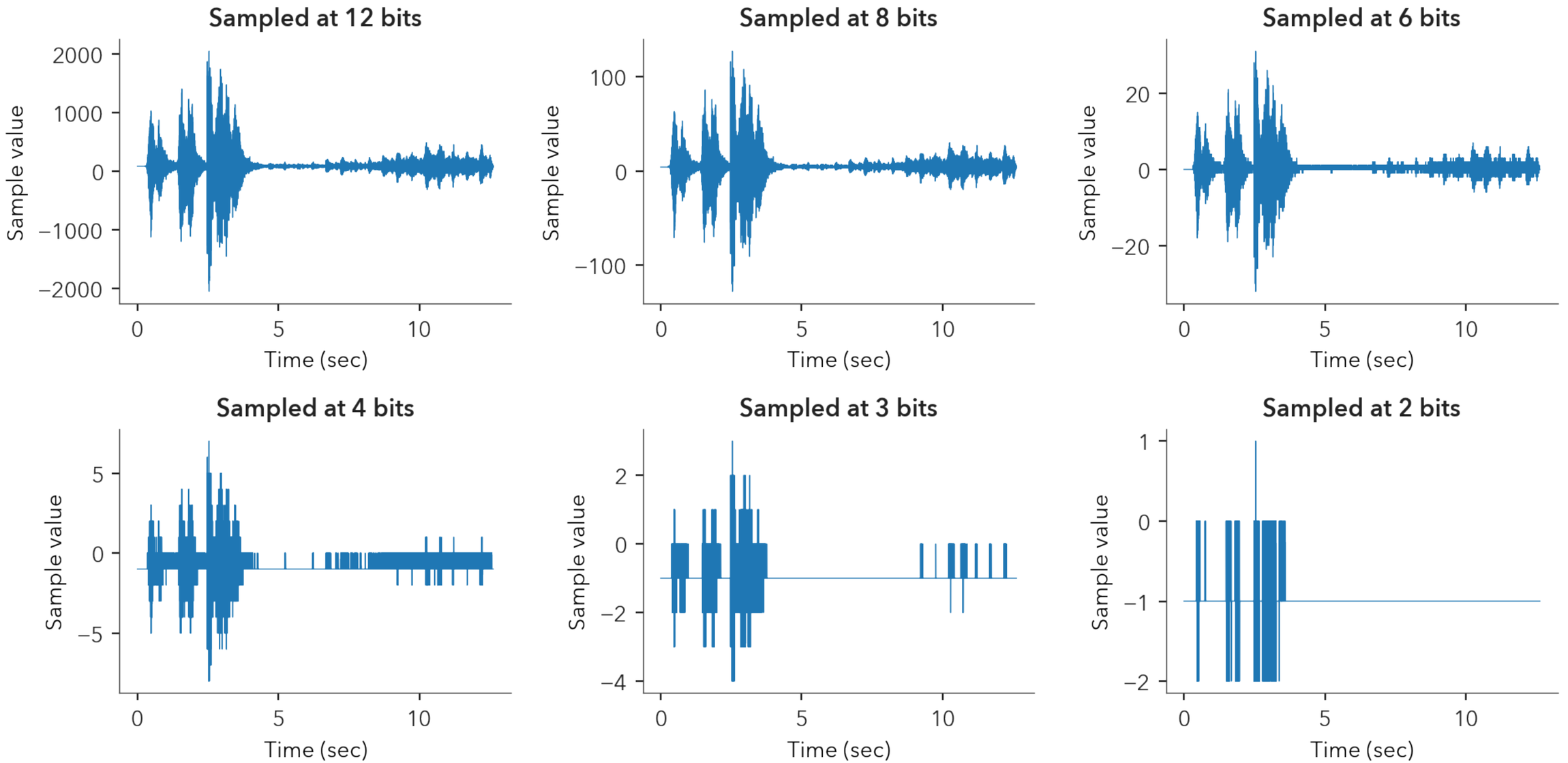
- Common ranges (*headroom*)
  - 16-bit / 96 dB (the industry standard)
  - 12-bit / 72 dB (the cheap standard)
  - 8-bit / 48 db (the 80's standard! hipsters?)
  - 24-bit / 144 dB (the "I'm charging you extra" standard)
  - Floating point (what we will use in class)

What more bits give us

- Need headroom to avoid *clipping* & *quantization noise*
  - These happen when the representation is maxed or zero
    - Very challenging with dynamic content (e.g. classical music)
      - An audio engineer's nightmare!

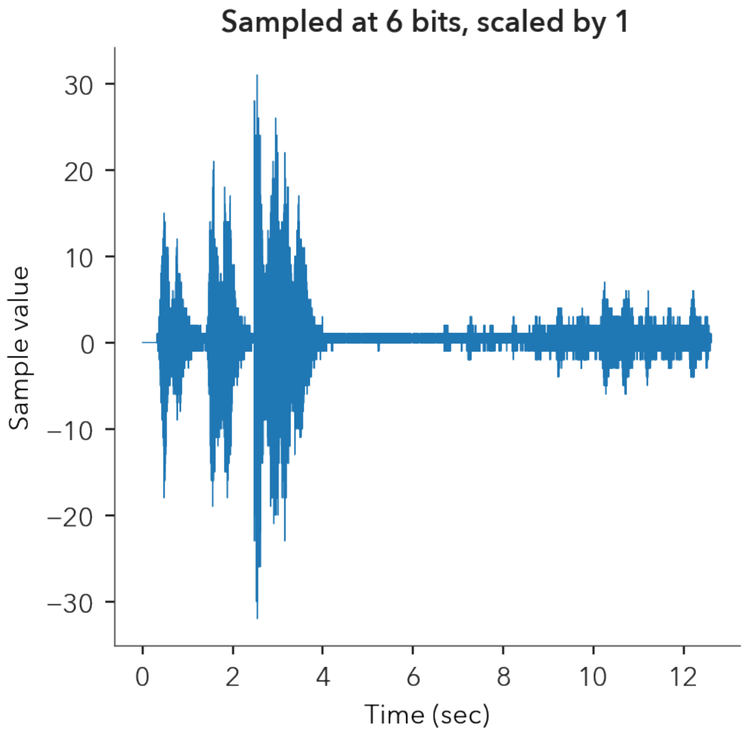


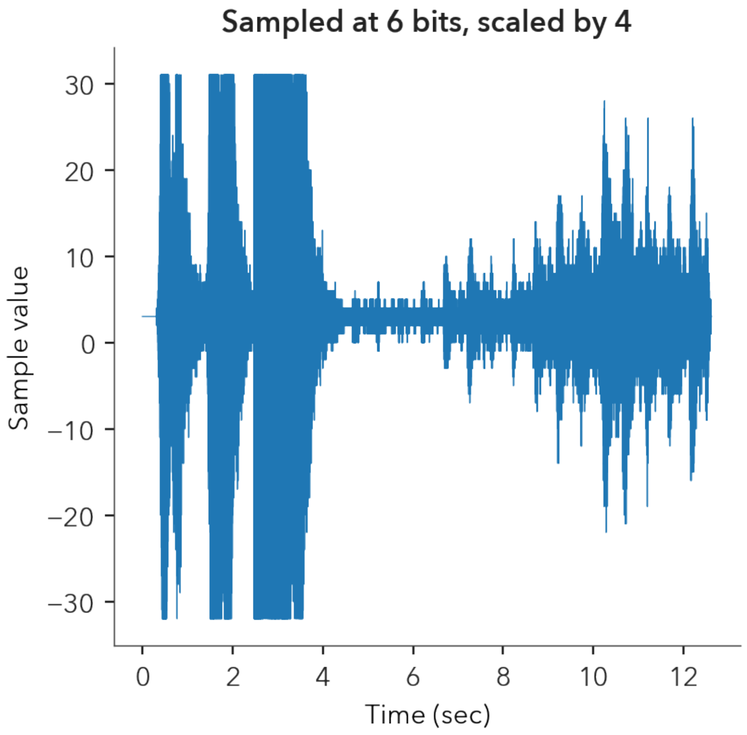
How does quantization sound?

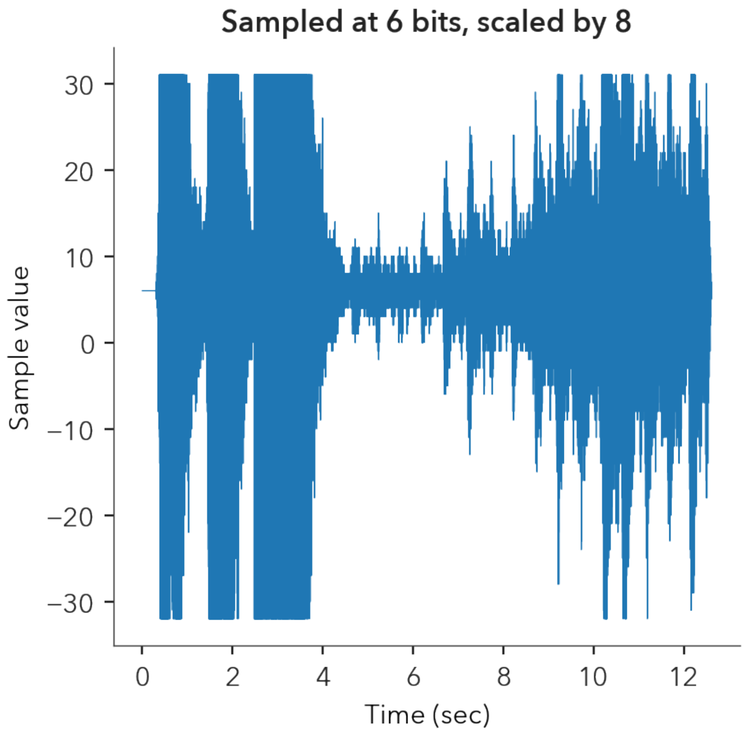


A risky fix

- We can boost the signal before we quantize so that the soft parts are not over-quantized
  - But then the loud parts get clipped





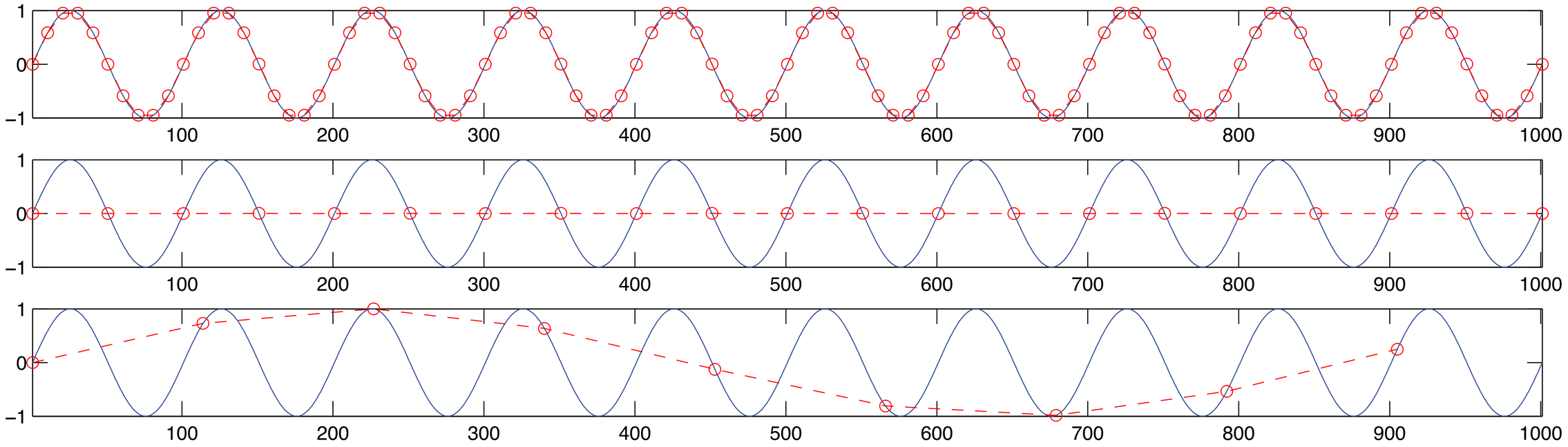


Sampling in time

- Also known as A/D conversion (or ADC)
  - How do we convert a real-world sound to a discrete sequence?
- The one parameter we care for: the *sample rate*
  - i.e. how often do we represent the input sound
- Tradeoffs
  - Sample fast and you waste memory and energy
  - Sample slow and you risk *aliasing*

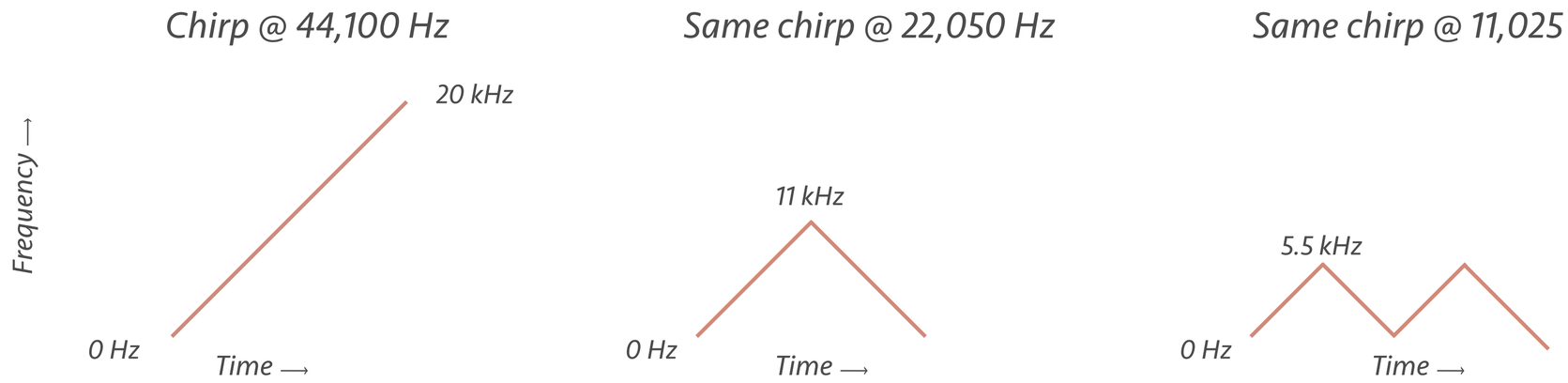
What is aliasing?

- Low sample rates can result in misinterpretations
  - Sample too low and you will miss some of the action
  - Rule of thumb: Sample at least at twice the highest frequency



What does aliasing sound like?

- Frequencies higher than *Nyquist* "fold over"
  - Upwards movements go downwards and vice-versa

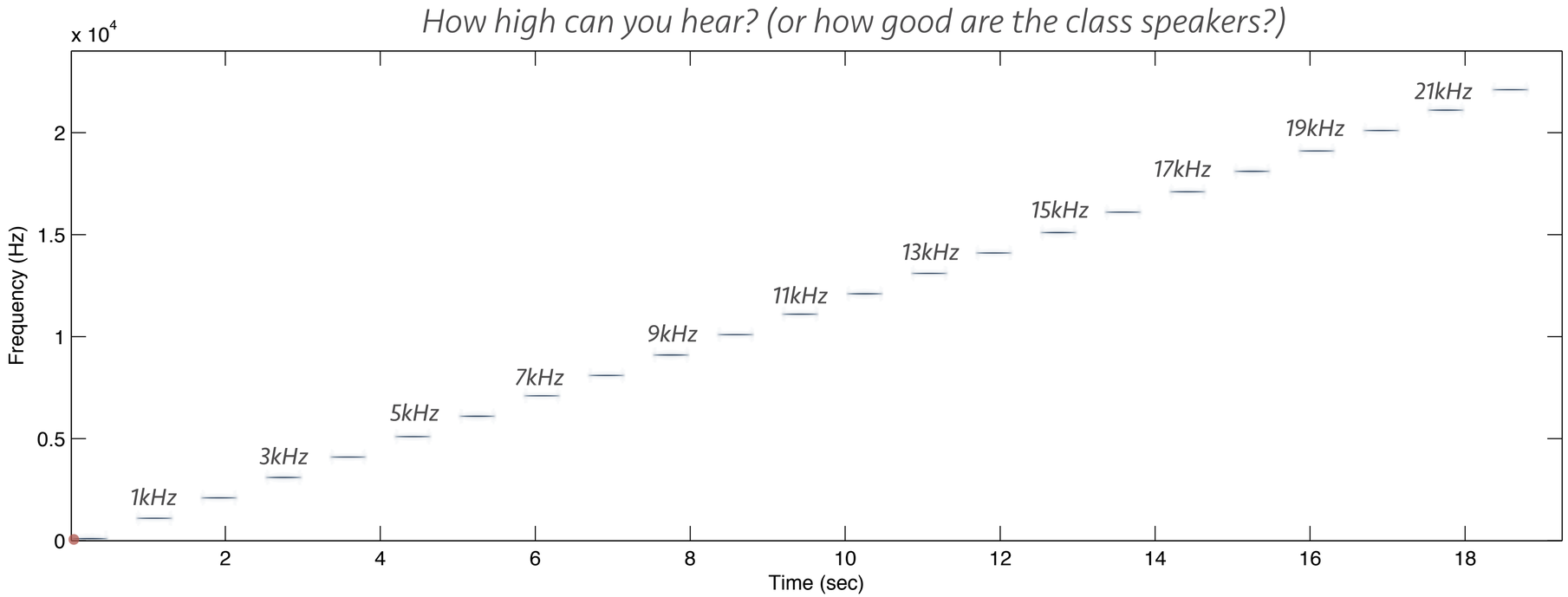


- Most noticeable with high-frequency content
  - How does that sound?

at 44.1kHz *at 22kHz* *at 11kHz* *at 5kHz* *at 4kHz* *at 3kHz*

How high should we go?

- Highest perceived frequency by humans is 20 kHz
  - Which goes down as you age (or as you abuse your ears)



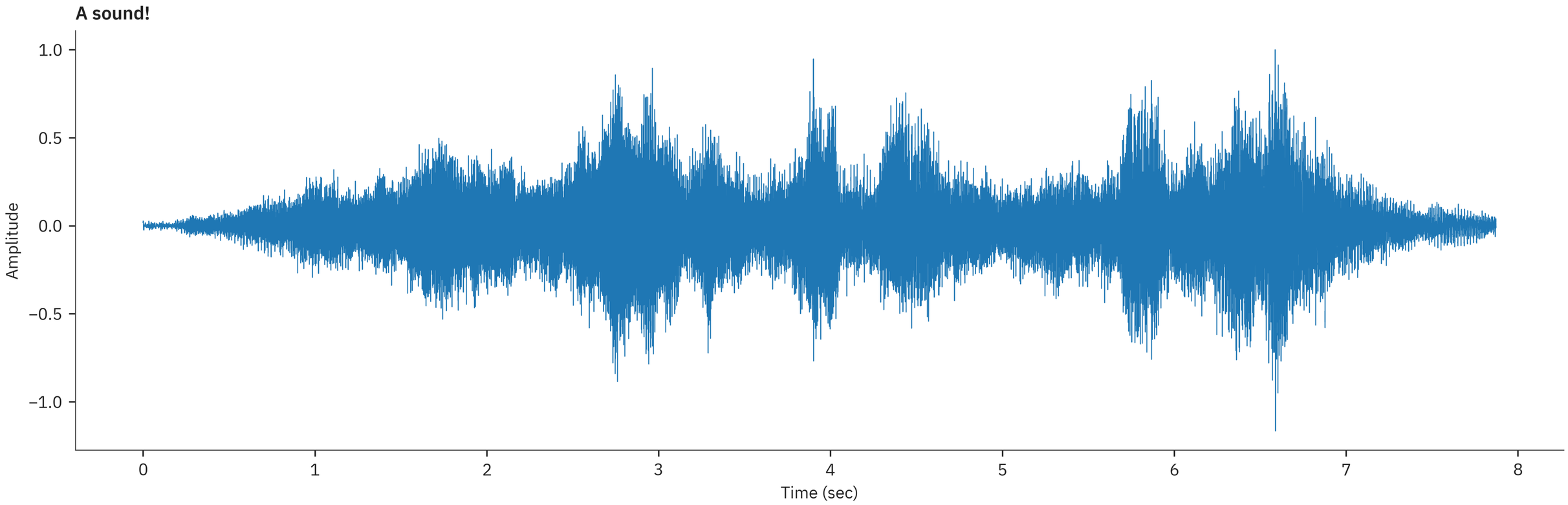
  - We need to represent up to 20 kHz ⟶ sample at > 40 kHz

What are the usual settings?

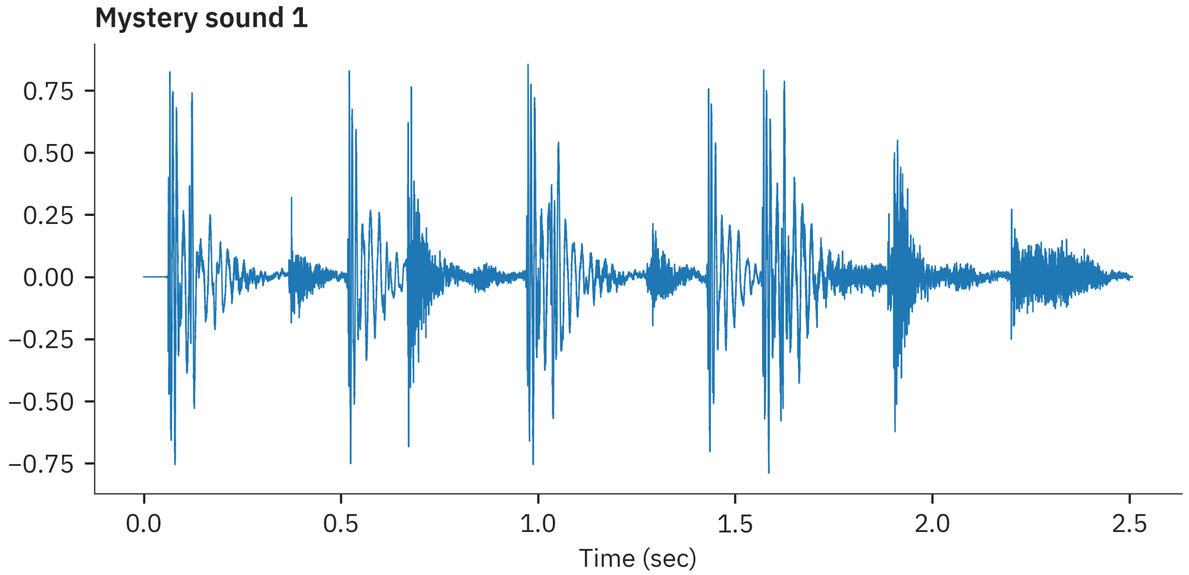
- “High-quality” music: 44.1 kHz
  - Why the extra 4.1 kHz?
- “Super” high quality music: 96 kHz
  - Dogs might like it more 🤷‍♂️
- Speech coding
  - High(ish) quality & for research: 16 kHz
  - Telephony: 8 kHz

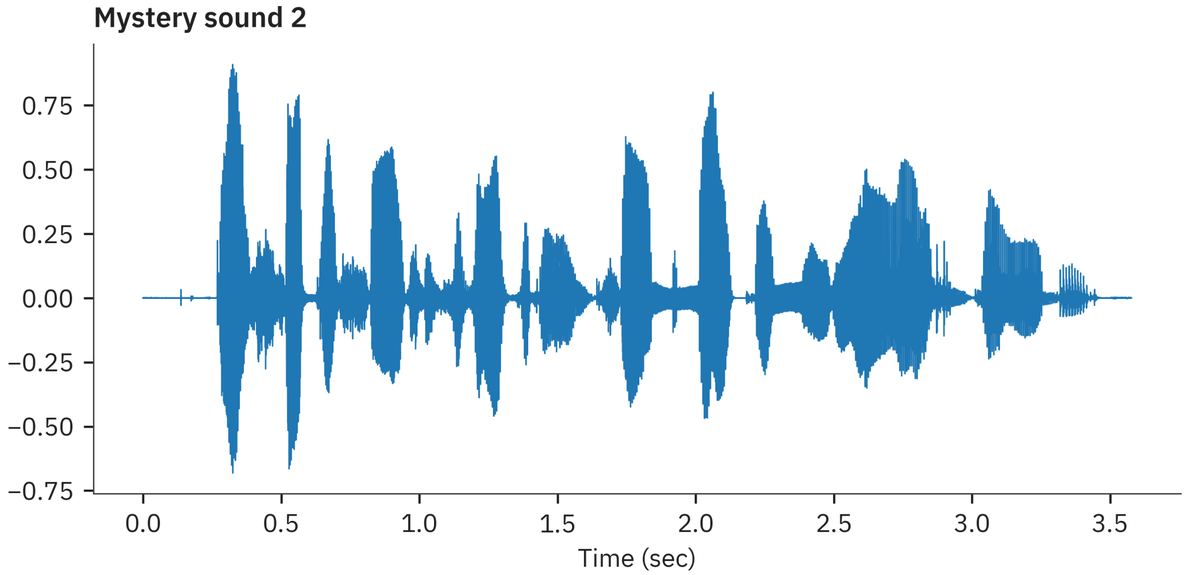
But why do we use the waveform?

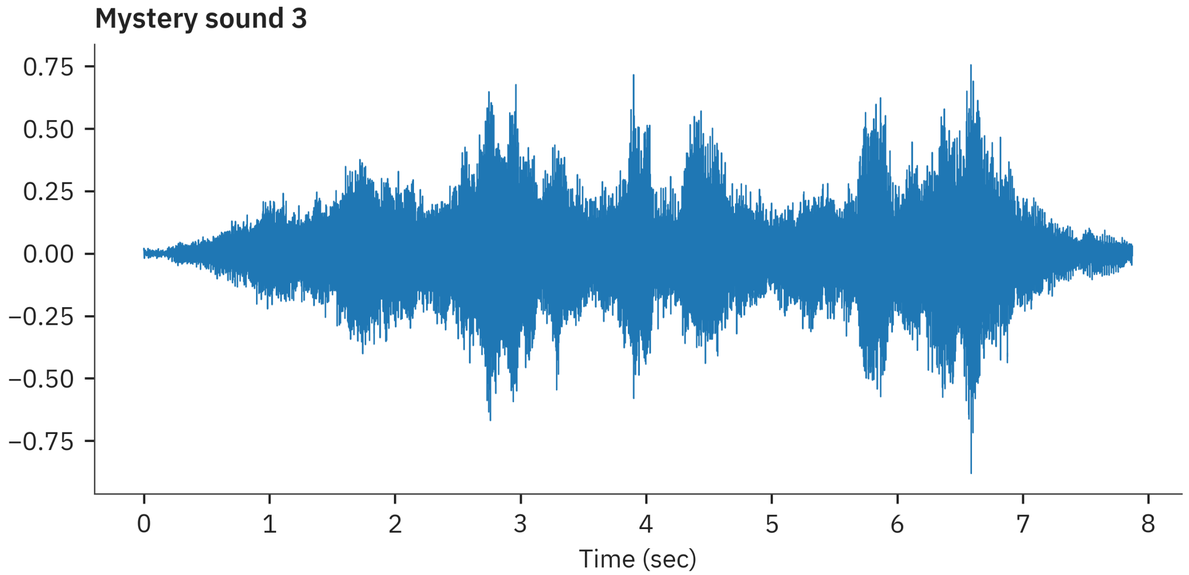
- Do you see a problem with it?

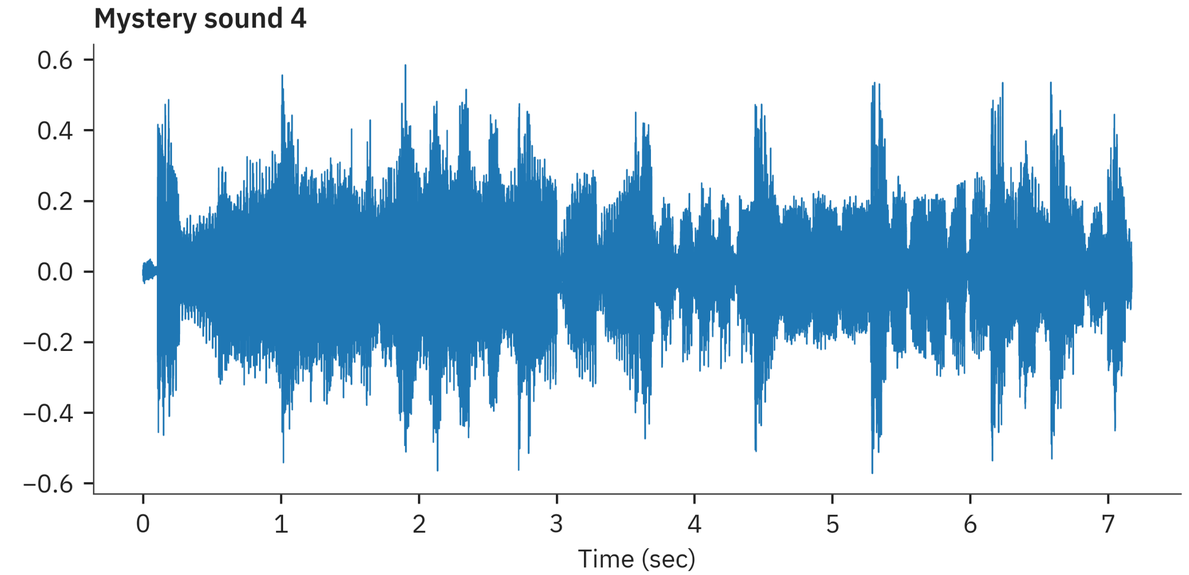


What are these signals?









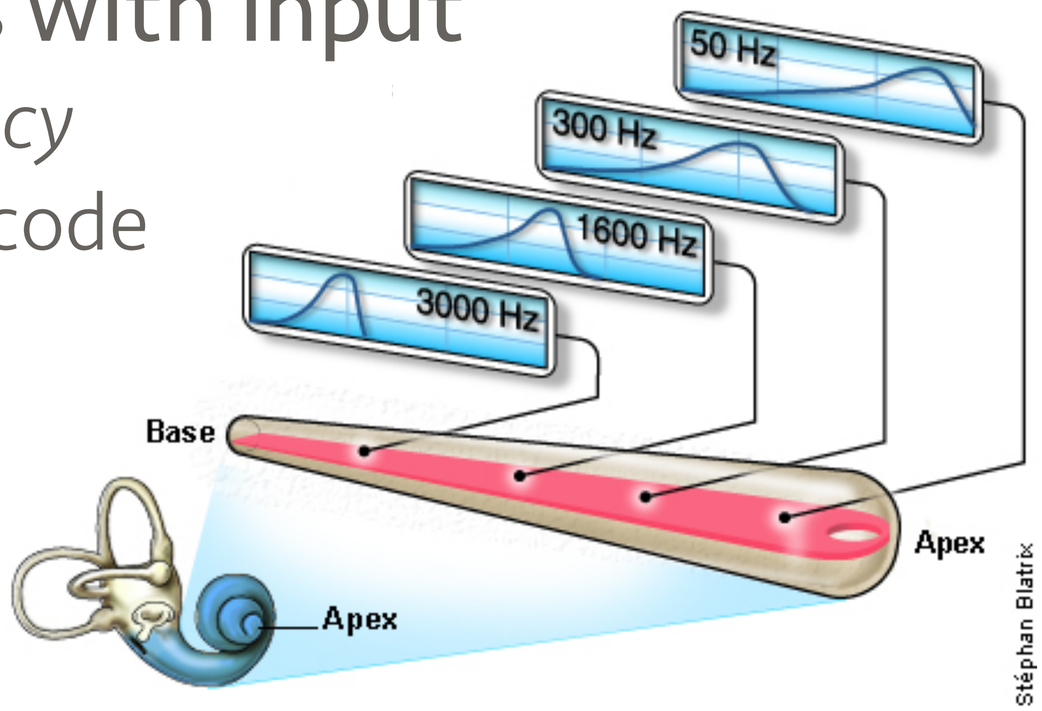
Waveforms are not intuitive at long scales

- Pressure information isn't perceptually meaningful
  - We cannot interpret it as a percept
  - Too much data to parse visually

- Is there a better way to represent sound?
  - How do we start looking for such a way?
  - What is it that is important when listening?

Back to hearing …

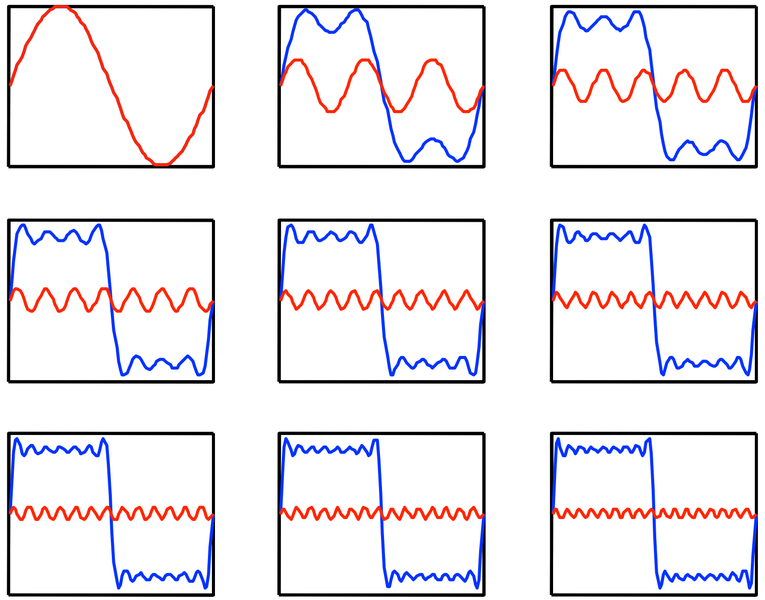
- What happens in the inner ear?
  - After the oval window there's the *cochlea*
- Resonates at different lengths with input
  - Effectively parses sound by *frequency*
  - Transmits that vibration to neural code
- What we care about is frequency content!



What is a frequency component?

- You can approximate any sound by adding some sinusoids
  - They are the elementary building blocks of sounds
    - Fast changes → high frequency content → high frequency sines
    - Smooth waveform → Low frequencies → low frequency sines
- Sinusoids have three parameters:
  - Amplitude, frequency and phase
  - $s(t) = a \sin(f t + \varphi)$
- Each sinusoid is a “frequency”
  - Because that is its primary distinguishing parameter (to our ears)

Approximating a square wave



Decomposing sounds to sums of sines

- How can we express a waveform with sines?
  - Need all amplitudes and phases for all possible frequencies
- For this we use the *Discrete Fourier Transform* (DFT)
  - Transforms time samples to the *frequency domain*, and back

"Spectrum" (frequency domain)

$$X[k] = \mathcal{F}(x[n])$$

Waveform (time domain)

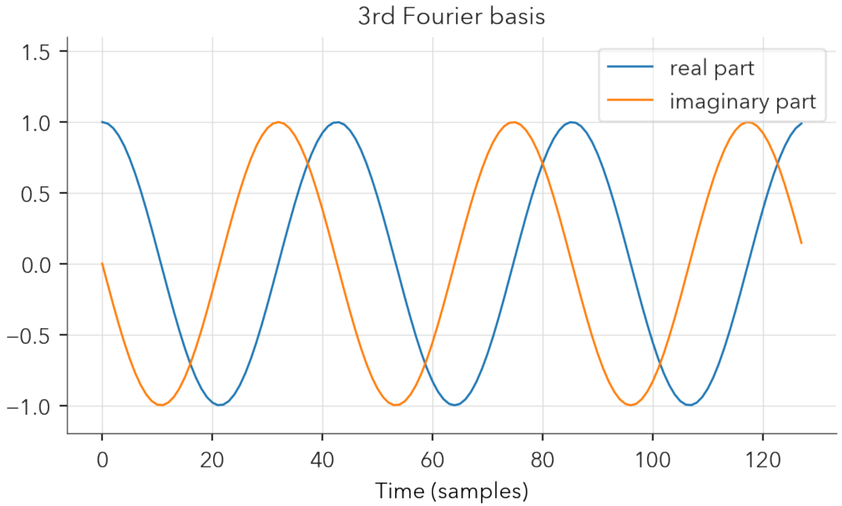
$$x[n] = \mathcal{F}^{-1}(X[k])$$

How does that work?

- Each Fourier basis contains a sine and a cosine

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i \frac{2\pi kn}{N}} =$$

$$= \sum_{n=0}^{N-1} x[n] \left[\cos\left(\frac{2\pi kn}{N}\right) - i \sin\left(\frac{2\pi kn}{N}\right) \right]$$

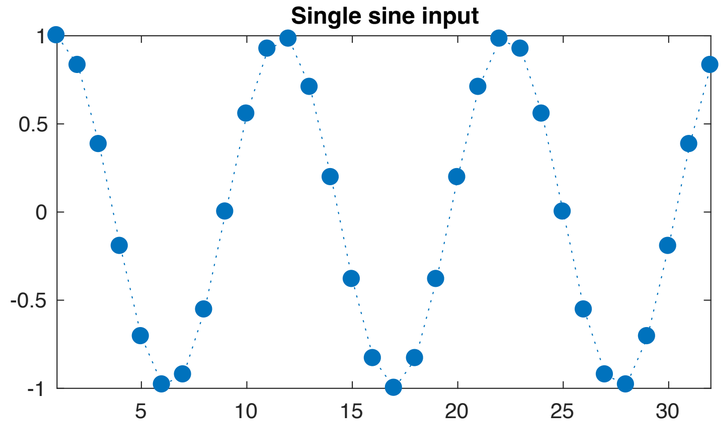


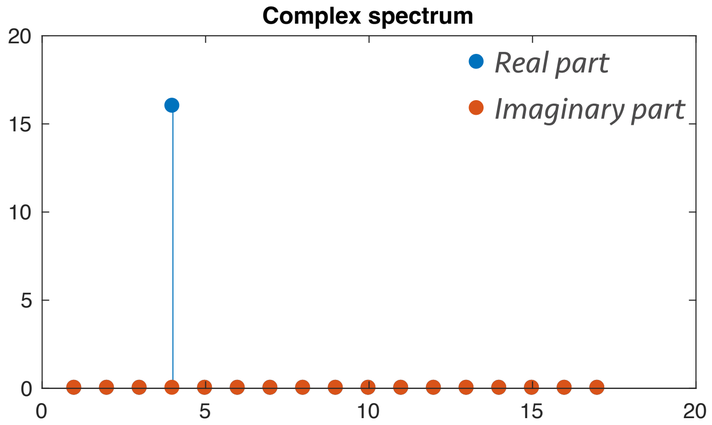
- The summation estimates how much of each cos/sin we have in the input time series (inner product)
  - Thus we get the contribution from each frequency

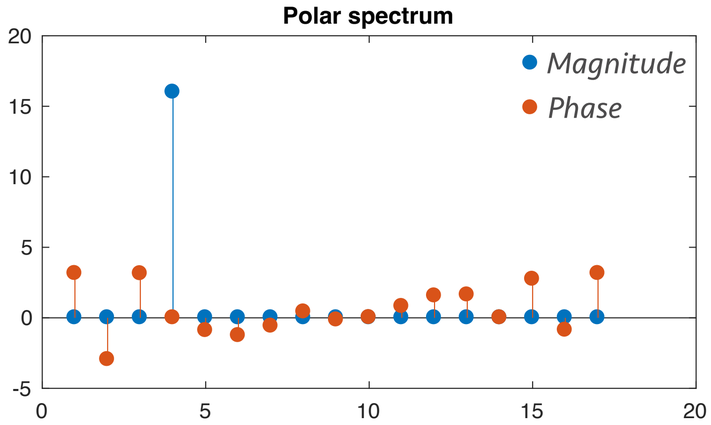
Getting to the stuff that matters

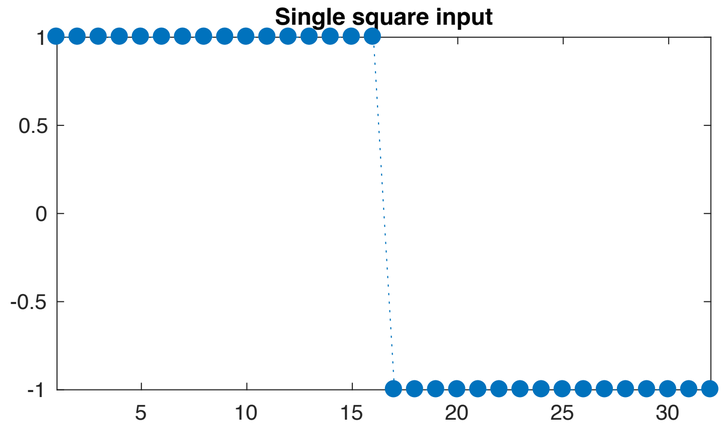
- The Fourier output is complex-valued
  - But what we usually want are its *magnitude* and *phase*
- The *magnitude spectrum*: $\|X[k]\| = \sqrt{\mathcal{R}(X[k])^2 + \mathcal{I}(X[k])^2}$
  - Tells us how much of each frequency we have (amplitudes)
    - Adding the sine and cosine terms we make phase-shifted sinusoids
      - The contribution of each frequency is the amount of sine/cosine present
- The *phase spectrum*: $\angle X[k] = \tan^{-1} \frac{\mathcal{I}(X[k])}{\mathcal{R}(X[k])}$
  - Gives us each frequency's phase
    - By looking at the relative amplitudes of the same-frequency sine/cosine pairs

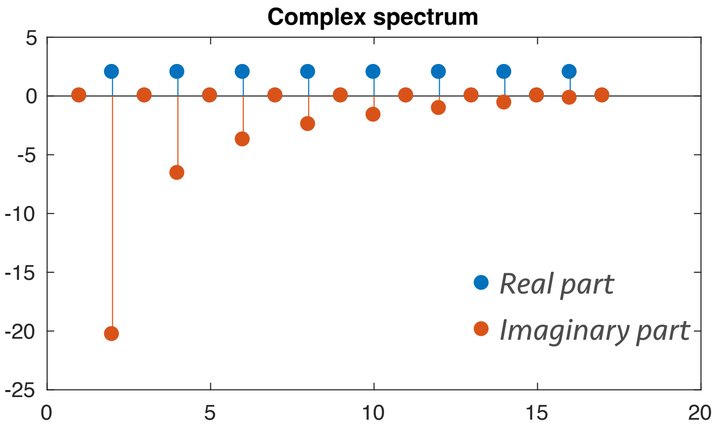
Some examples

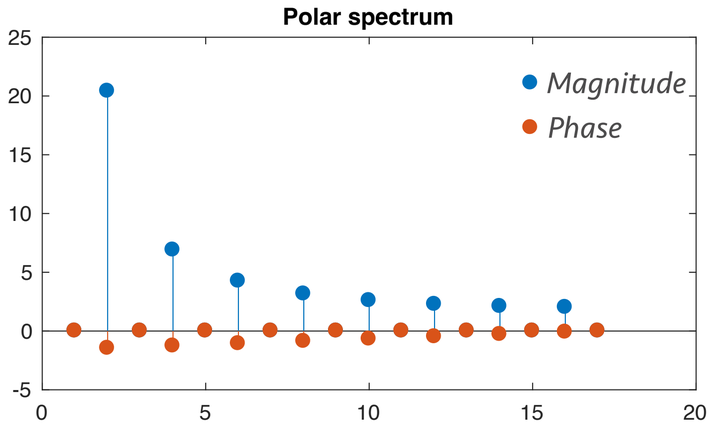






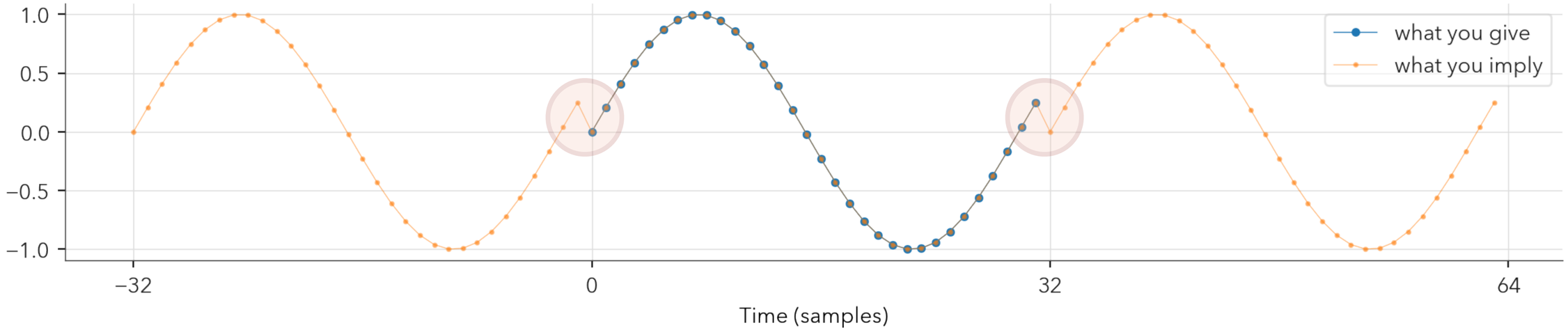






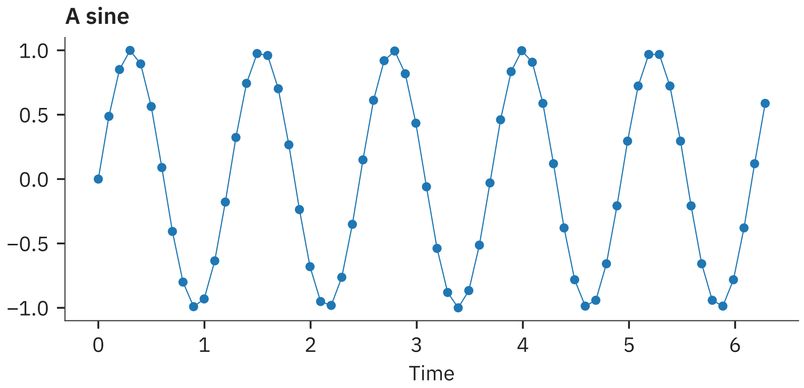
One problem

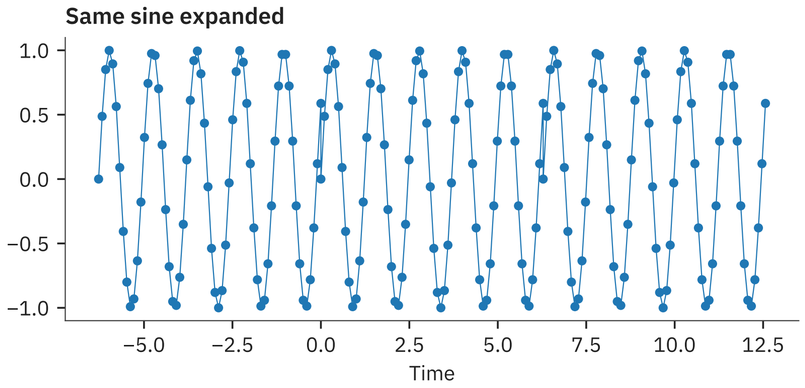
- Sinusoids extend infinitely on both sides
  - i.e. what we approximate is assumed to be periodic
  - So it should transition smoothly from left to right
- We need to ensure smoothness to get sensible results
  - Implicit discontinuities will result in high frequency content

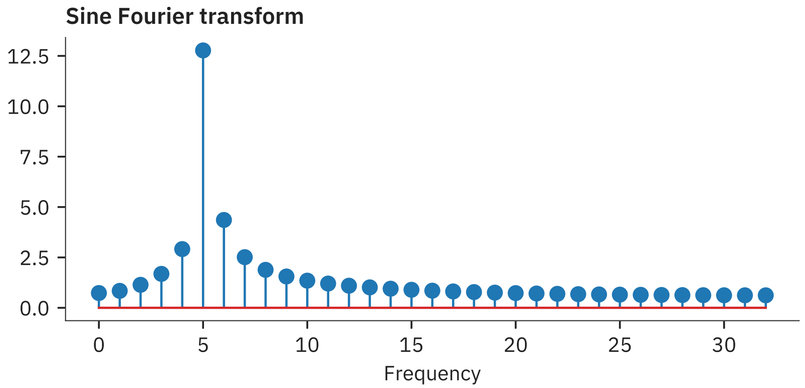


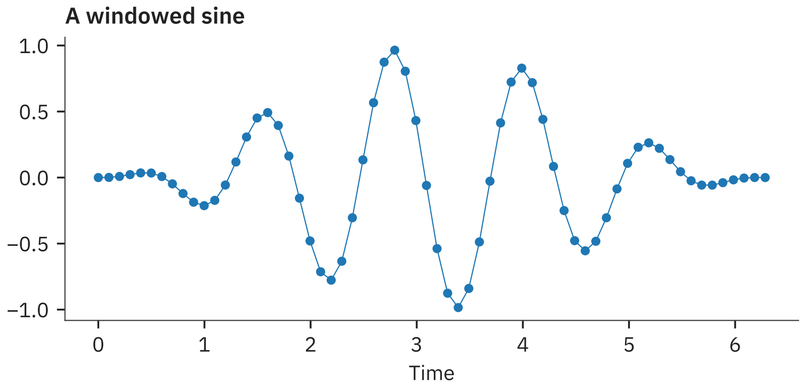
Windowing

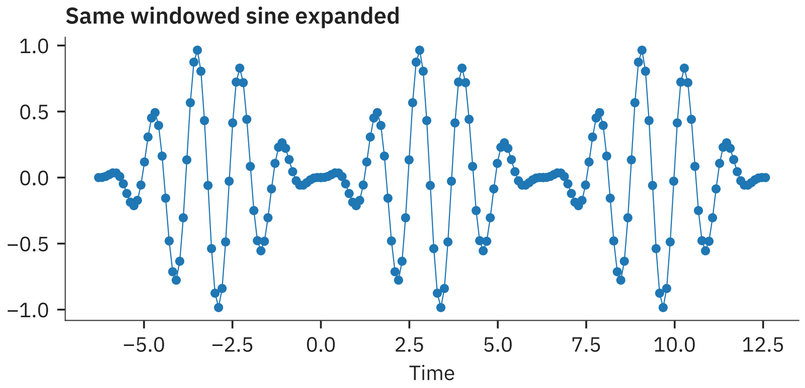
- To avoid periodic discontinuities we can “window”
  - Taper the ends to zero so that they join better
  - But too much can alter the signal — usually blurs the spectrum

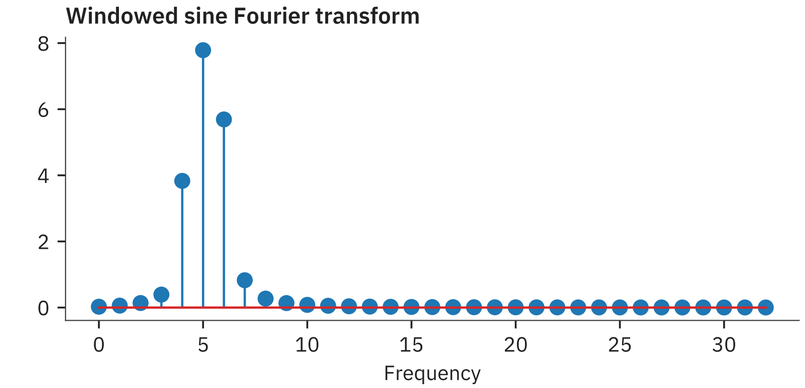






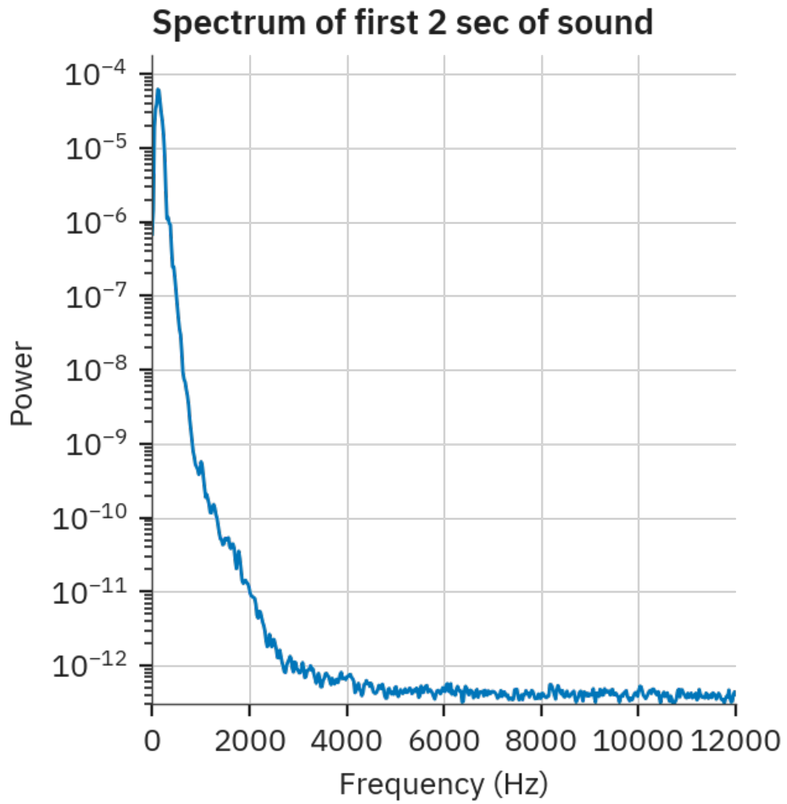


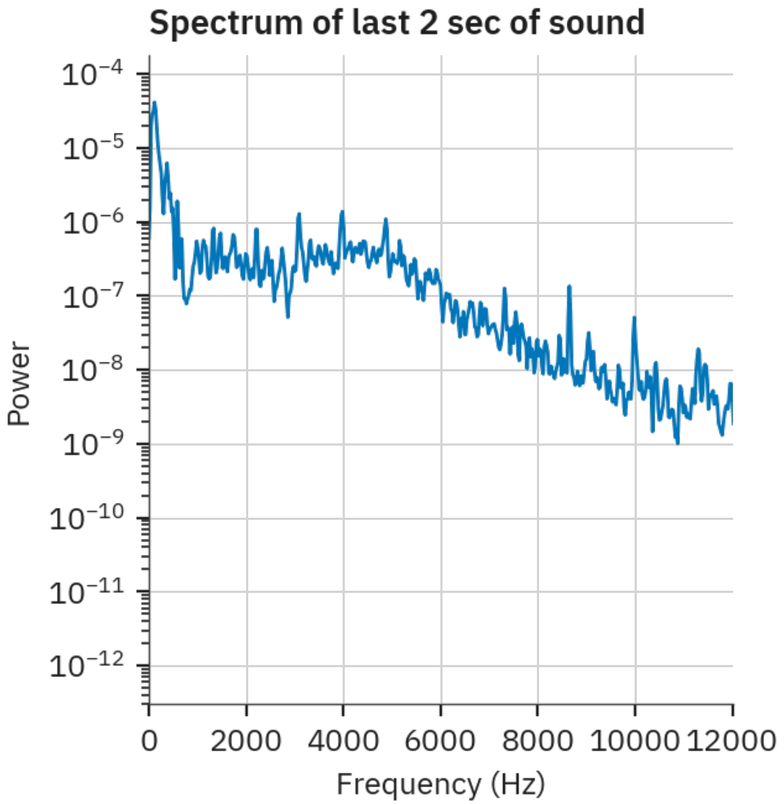


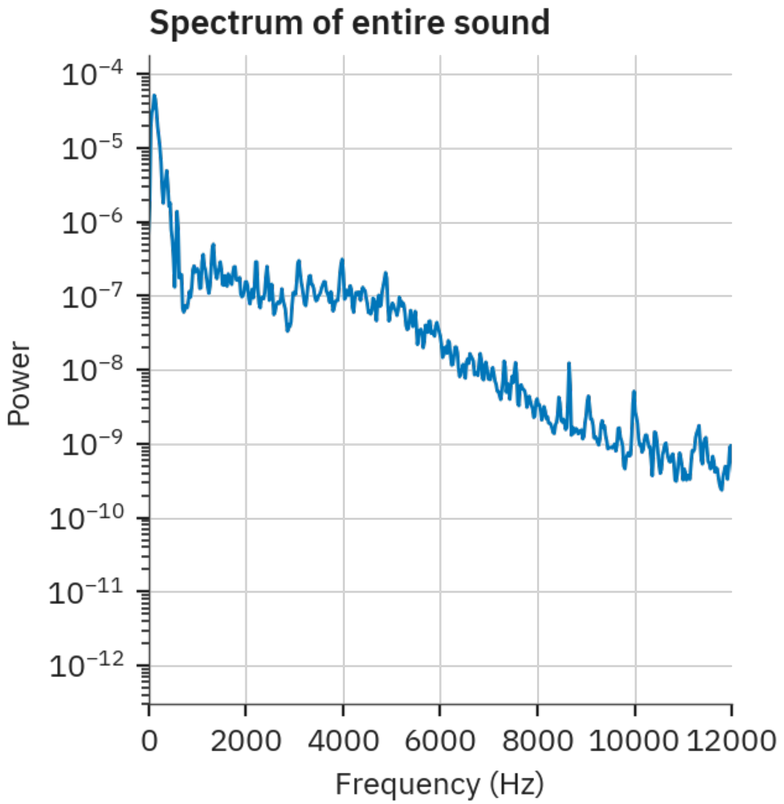


On real sounds

- We get a better sense of what's in a signal
  - But we are missing the time structure!







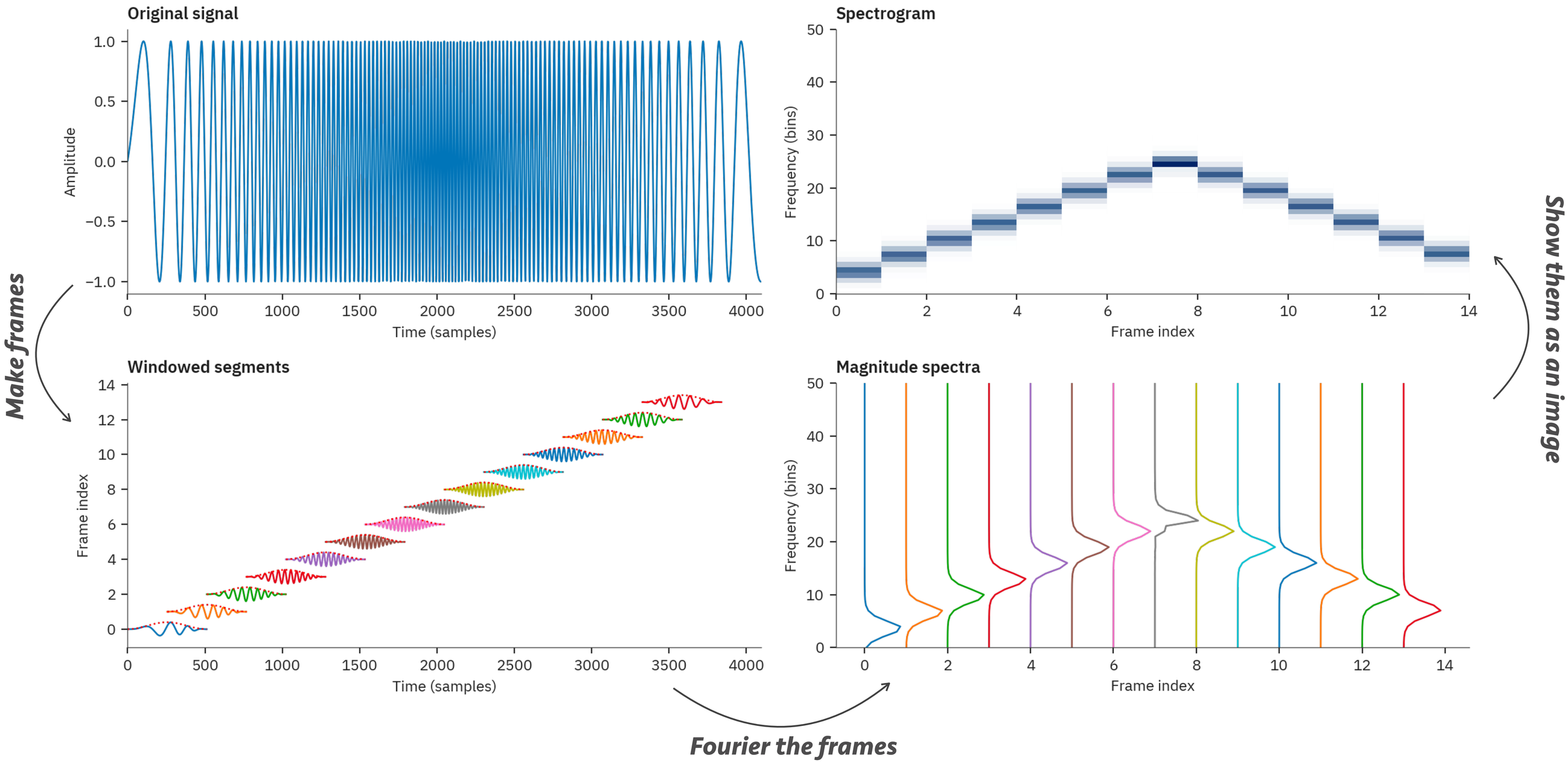
Adding a time dimension

- How about sampling spectra periodically?
  - Each sound segment will have it's own spectrum
    - "Short-time frequency analysis"
- Break input into analysis windows and Fourier them
  - Plot all successive spectra side by side
- Keeps time info, but also presents frequency content

Constructing a *spectrogram*

- Get N samples (a *frame*) and apply window
  - Tapers the edges makes for better estimate
- On windowed frame apply DFT
  - Gets frequency domain of this section only
  - DFT can also be M-points ($M > N$)
    - Input is zero-padded to length M
- Advance H samples, and repeat
  - H is the hop size
- Collate all spectra in 2D representation

Pretty picture version

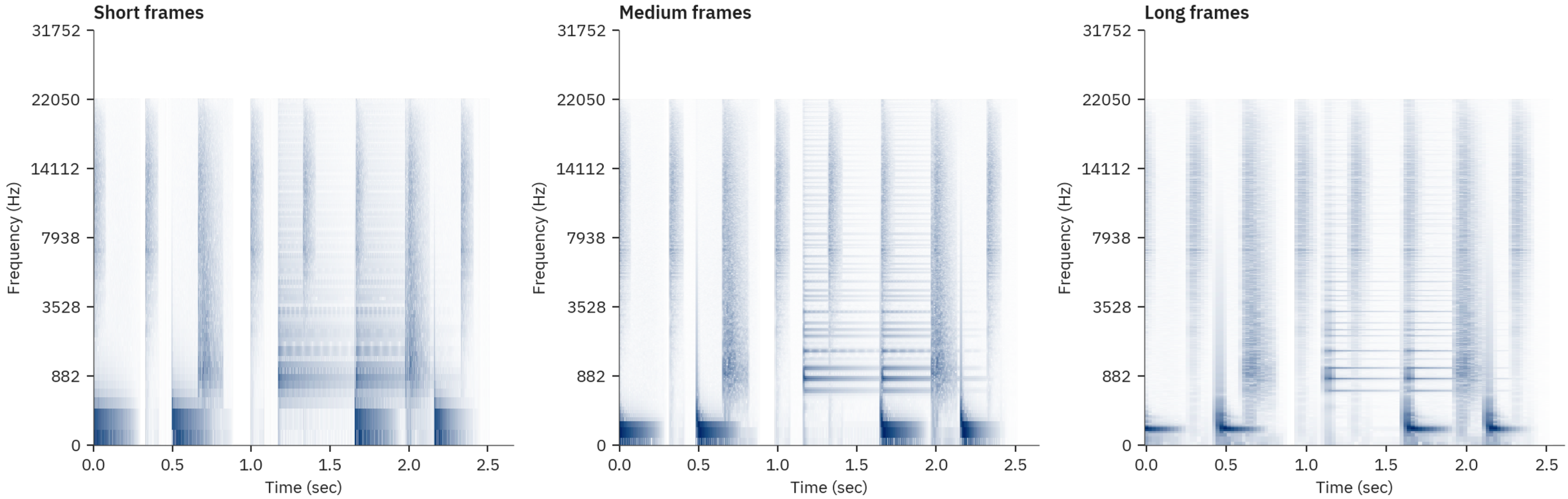


Spectrogram parameters

- Size of the Discrete Fourier Transform
  - Determines how fine the frequency resolution is
  - To get more frequencies we can zero pad
- Hop size
  - Determines how fine the temporal resolution is
  - Should not be larger than transform size! (why?)
- Window
  - Tradeoff between artifacts and frequency resolution
    - Stronger window more blurring, weaker window more artifacts

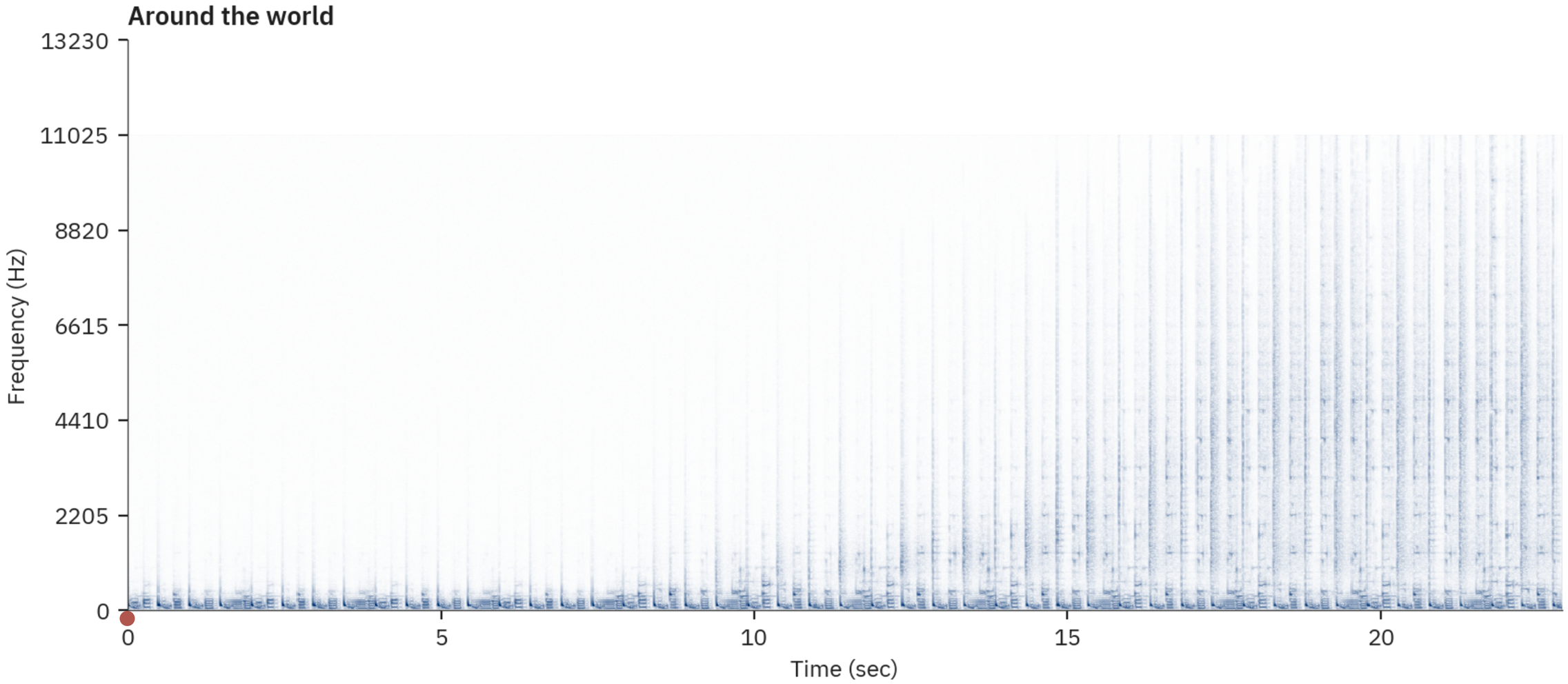
Time/frequency tradeoff

- The more frequencies, the fewer time points
  - And vice-versa

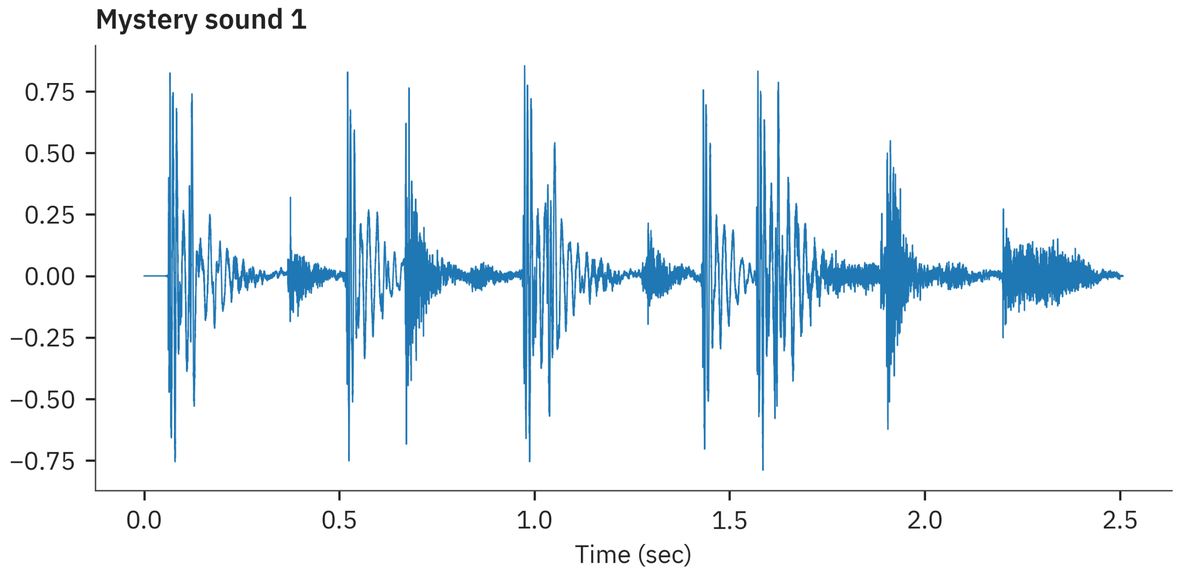


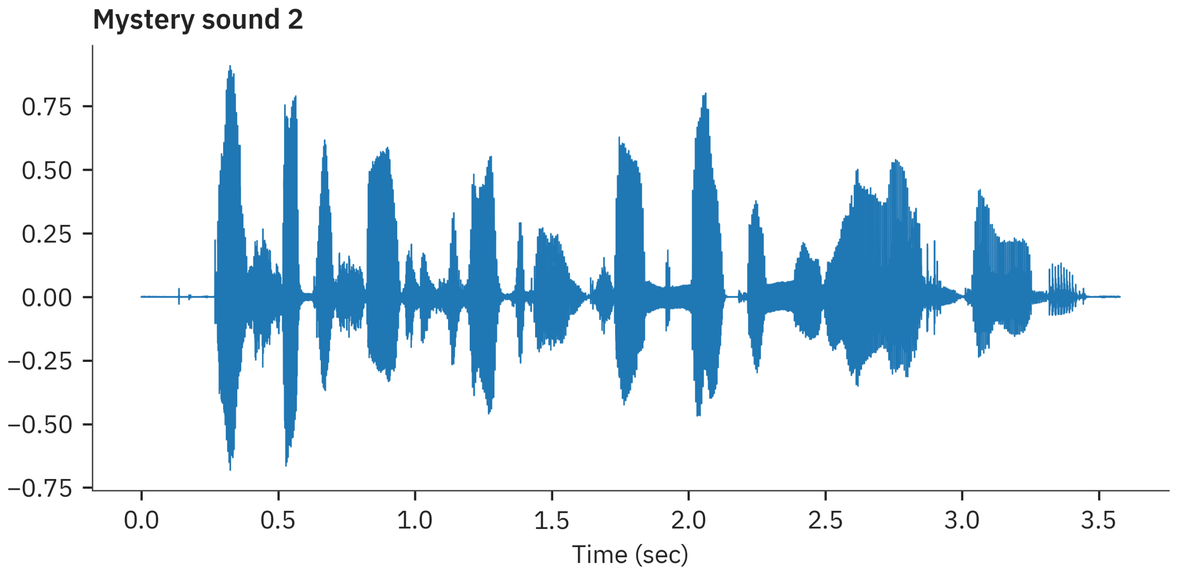
Back to the previous example

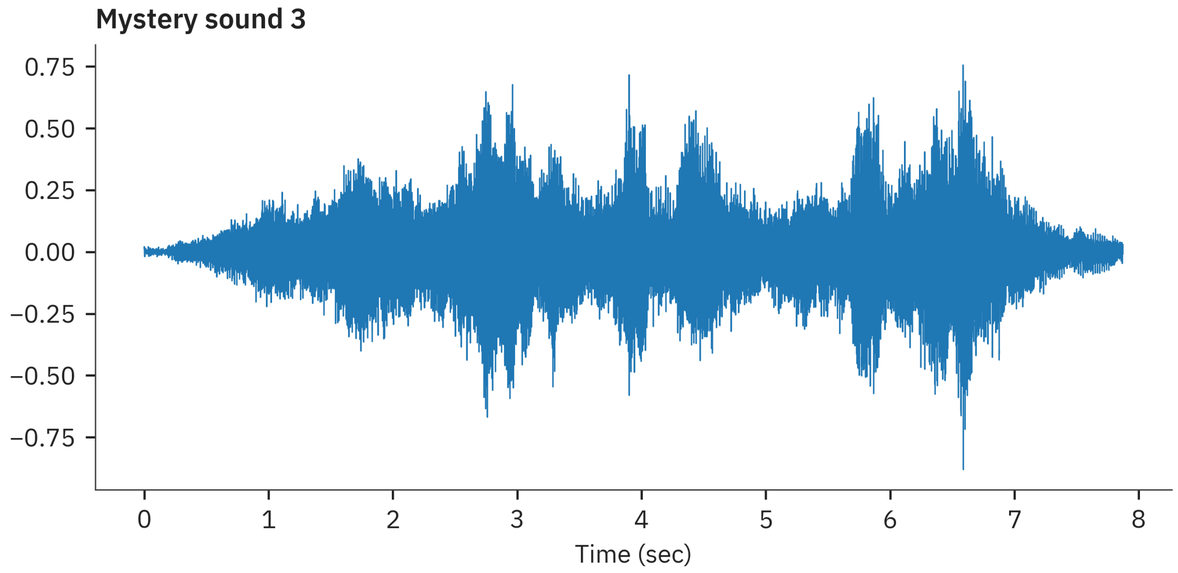
- With the spectrogram we can now see what goes on

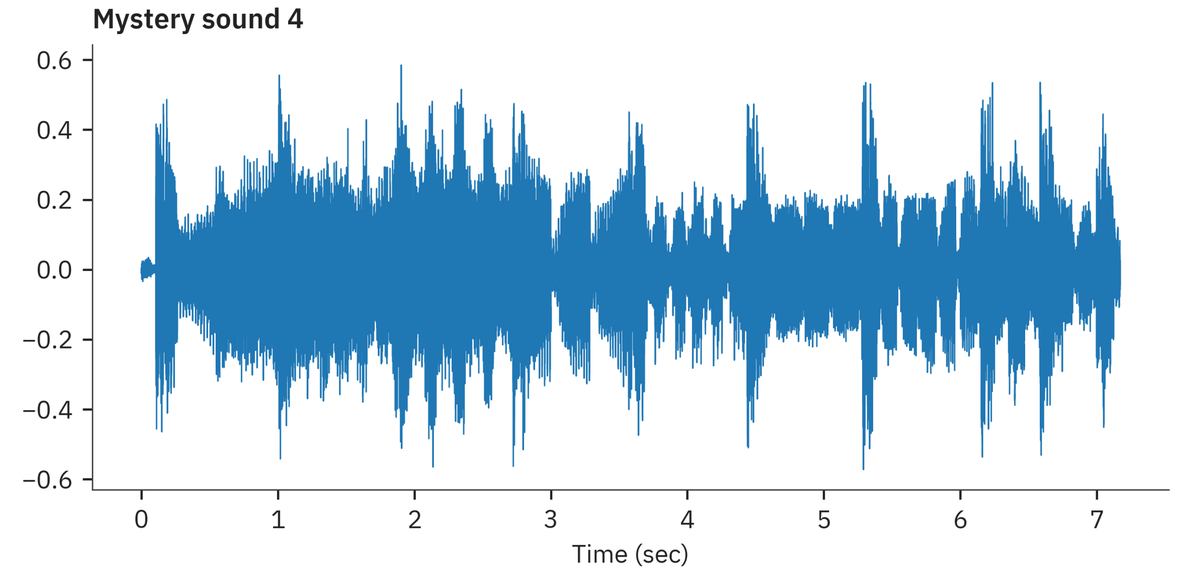


Remember these?

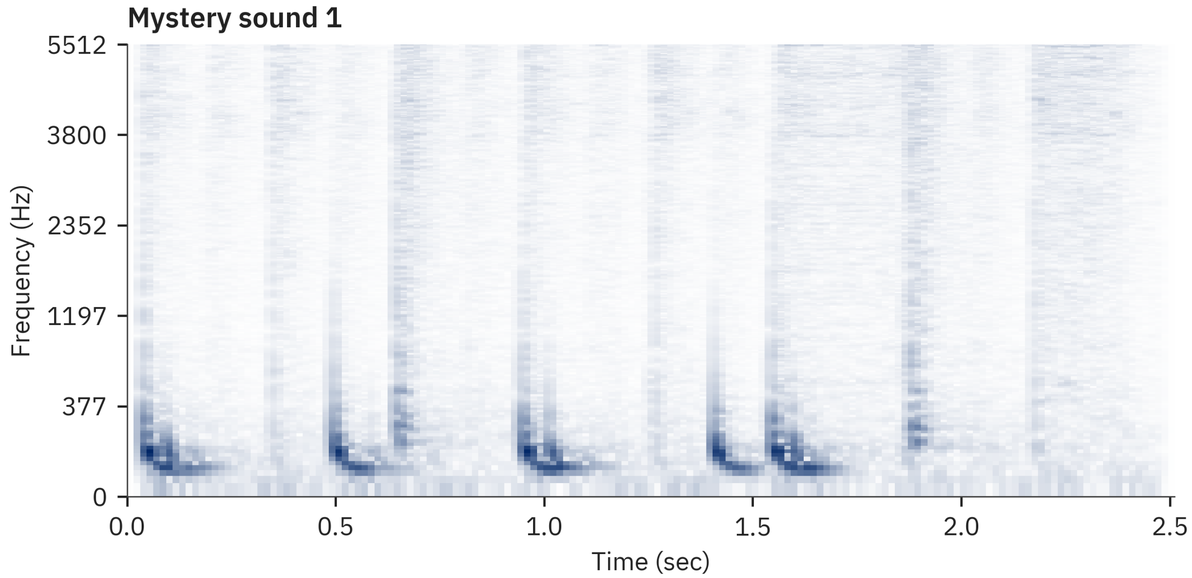


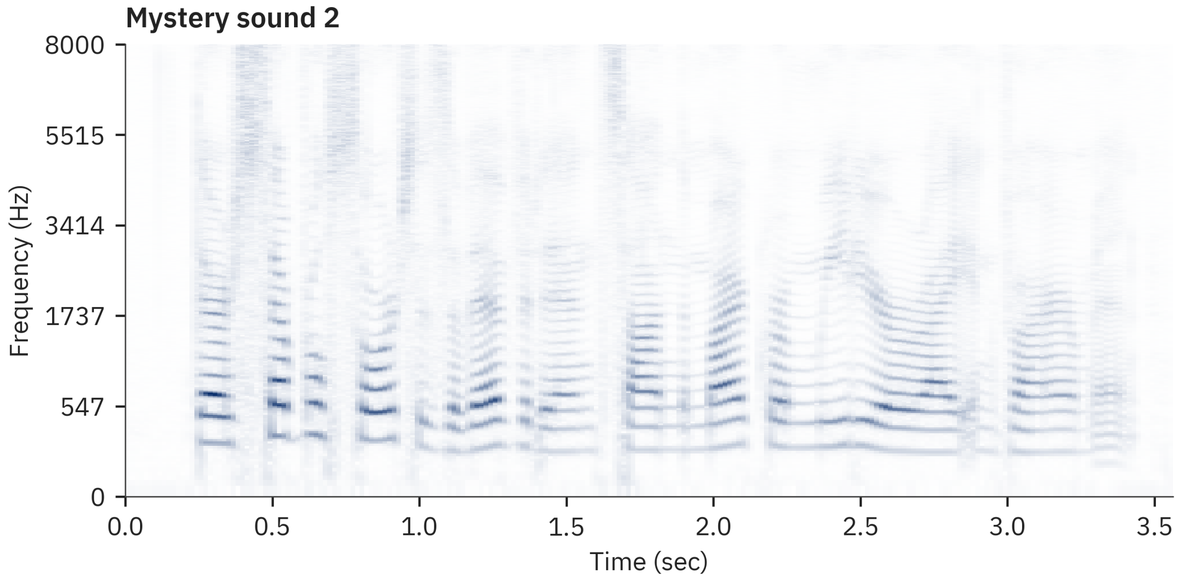


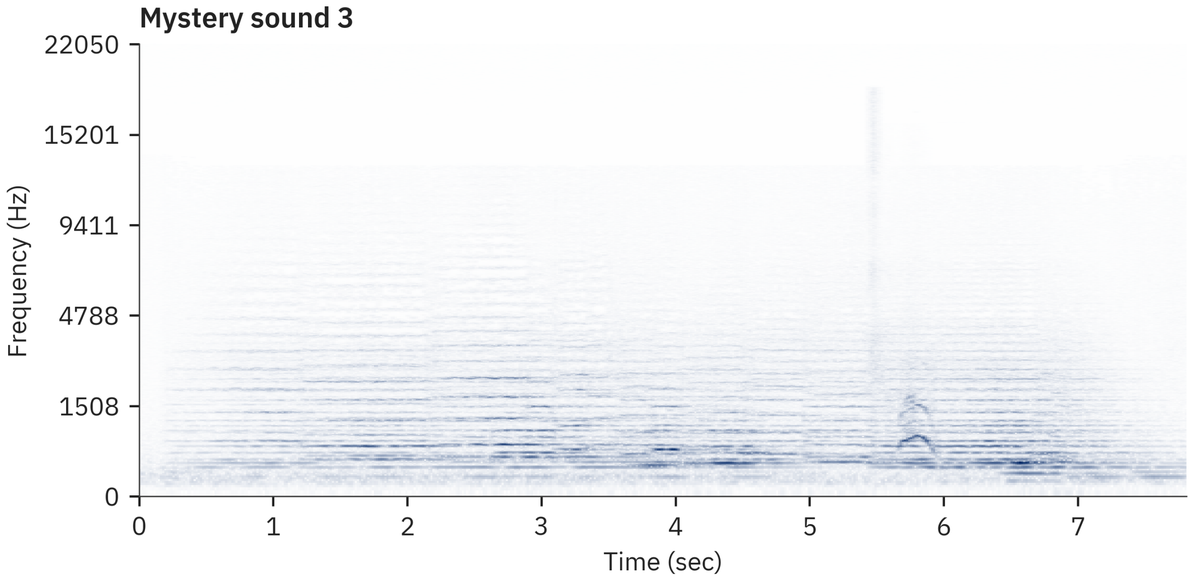


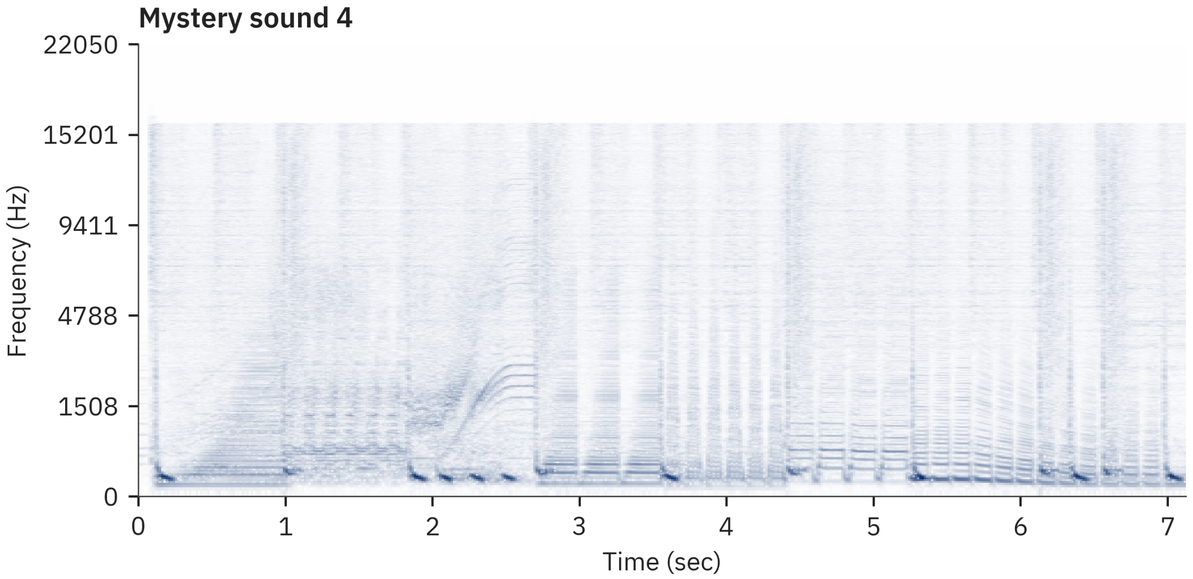


We can now “see” what goes on









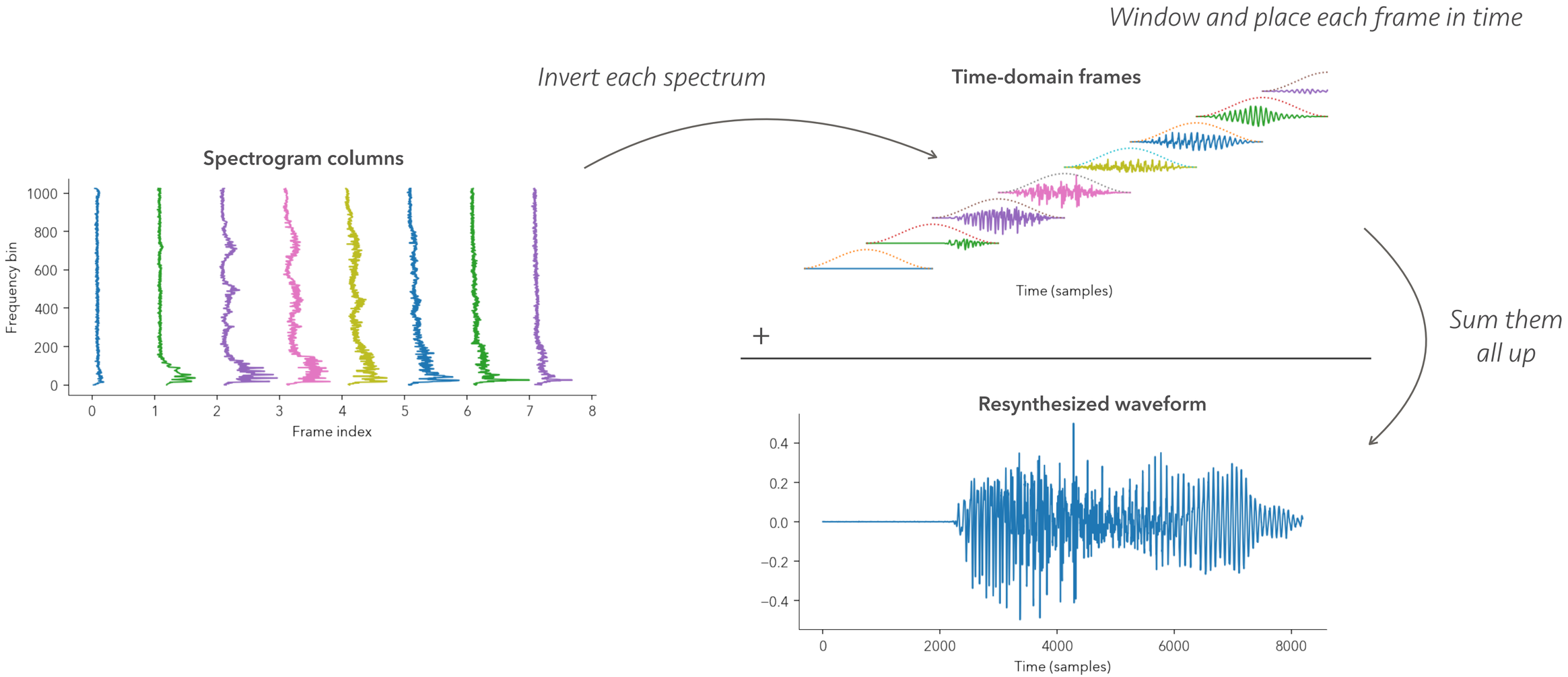
The inverse spectrogram

- We can also go from spectrogram to waveform
  - Inverting the spectrogram procedure

- For each (complex-valued) spectral frame
  - Convert to time segment using inverse DFT
  - Optionally apply a window again
  - “Overlap and add” segments that coincide

Pretty picture version

- Overlap-add procedure



Careful when inverse windowing!

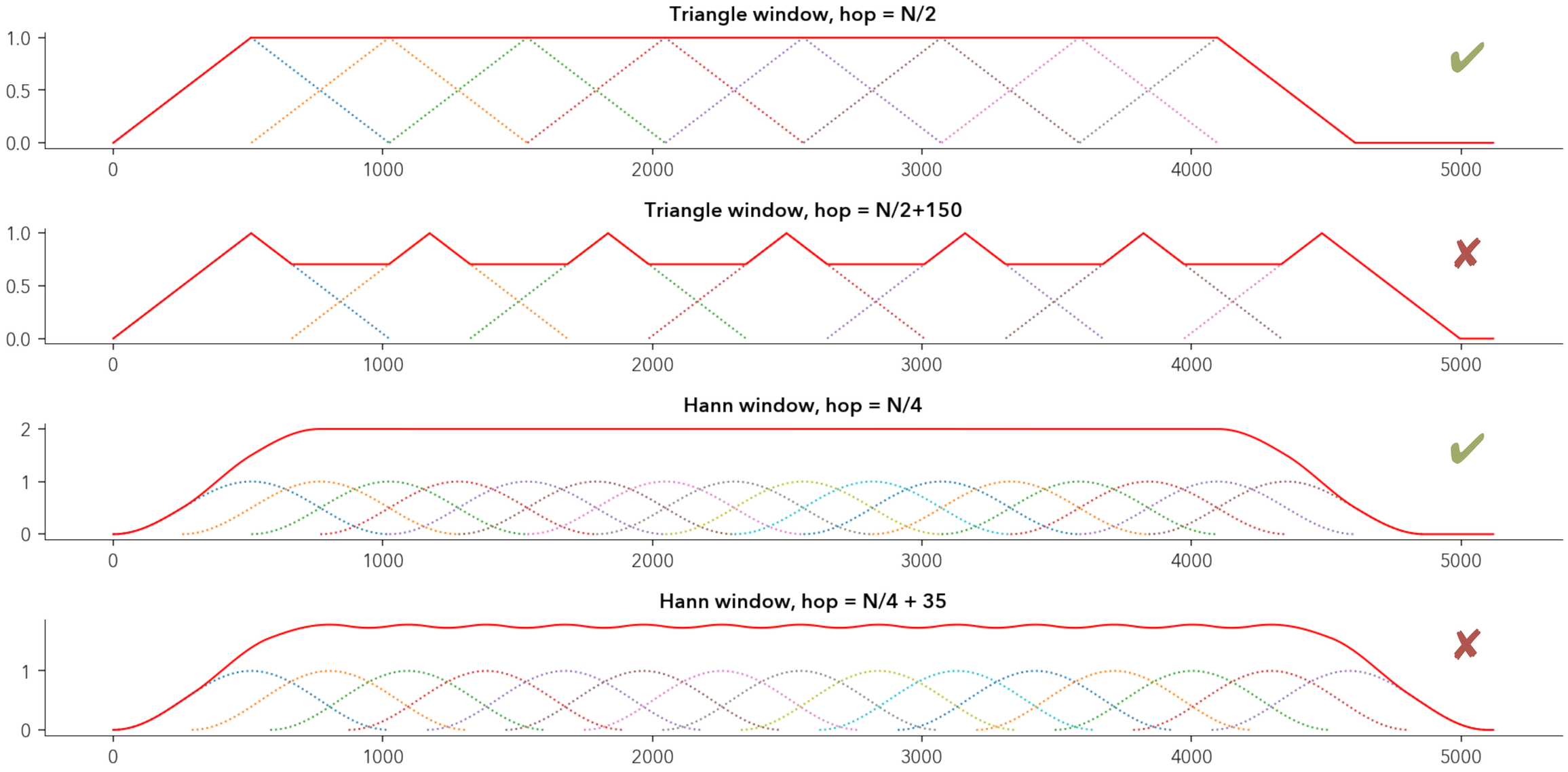
- If you use no windows the output scales with hop size
  - That's the number of times you overlap-add segments

- If you use windowing you need to satisfy COLA:
  - *Constant OverLap Add*:

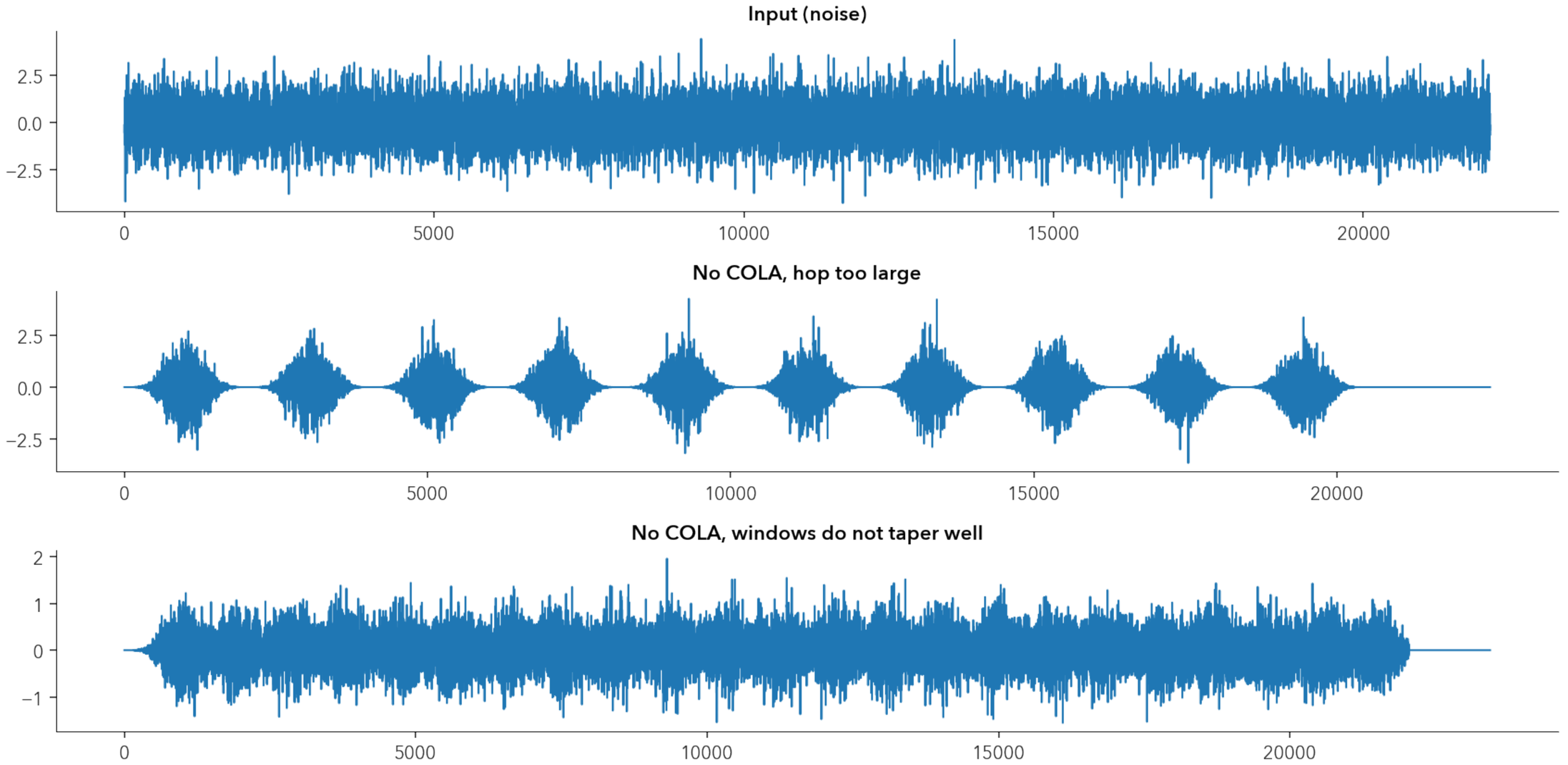
$$\sum_{m=-\infty}^{\infty} w(n - mH) = 1, \forall n \in \mathbb{Z}$$

  - where H is the hop size

What does that mean?



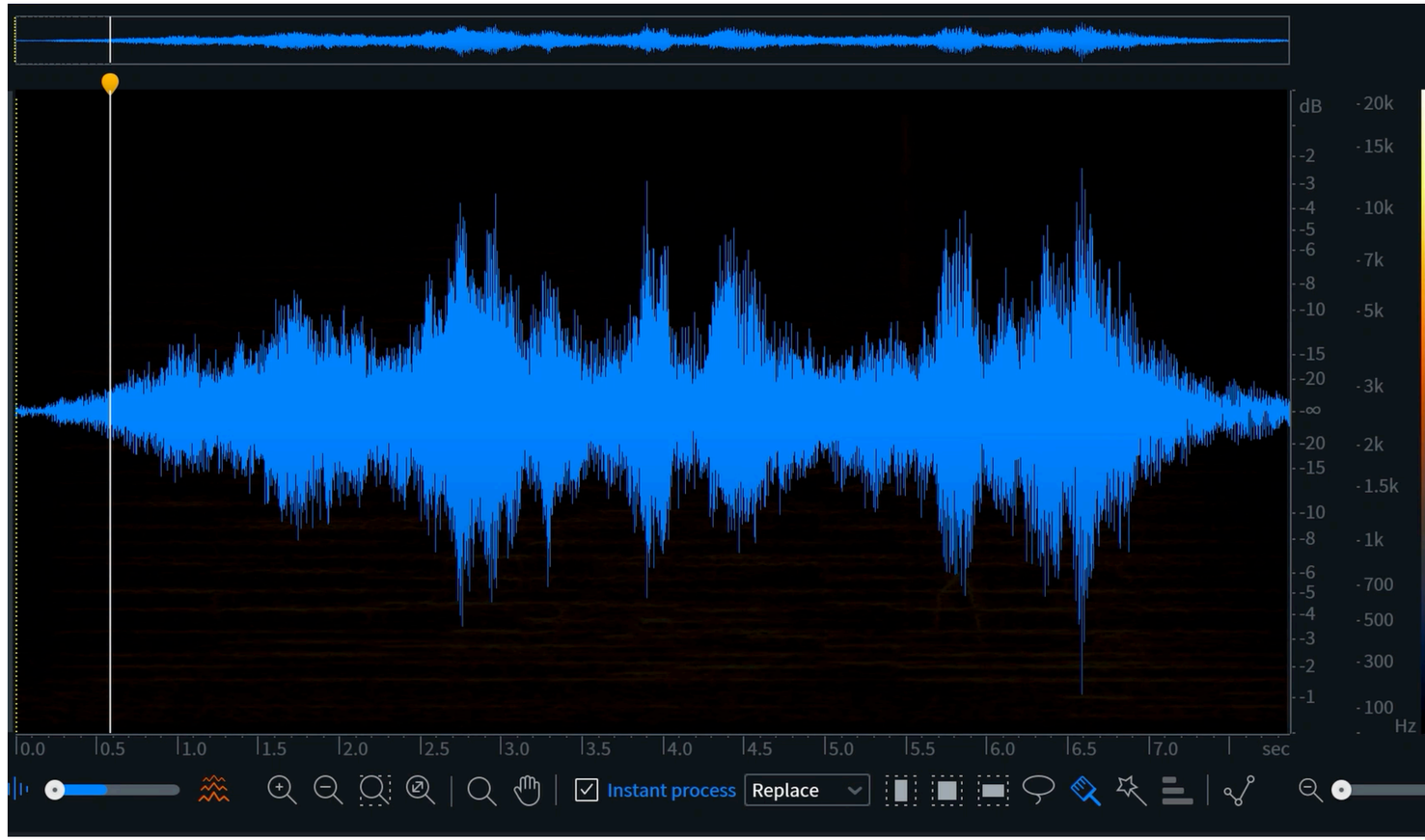
Examples of bad windowing



Some uses of inverse spectrograms

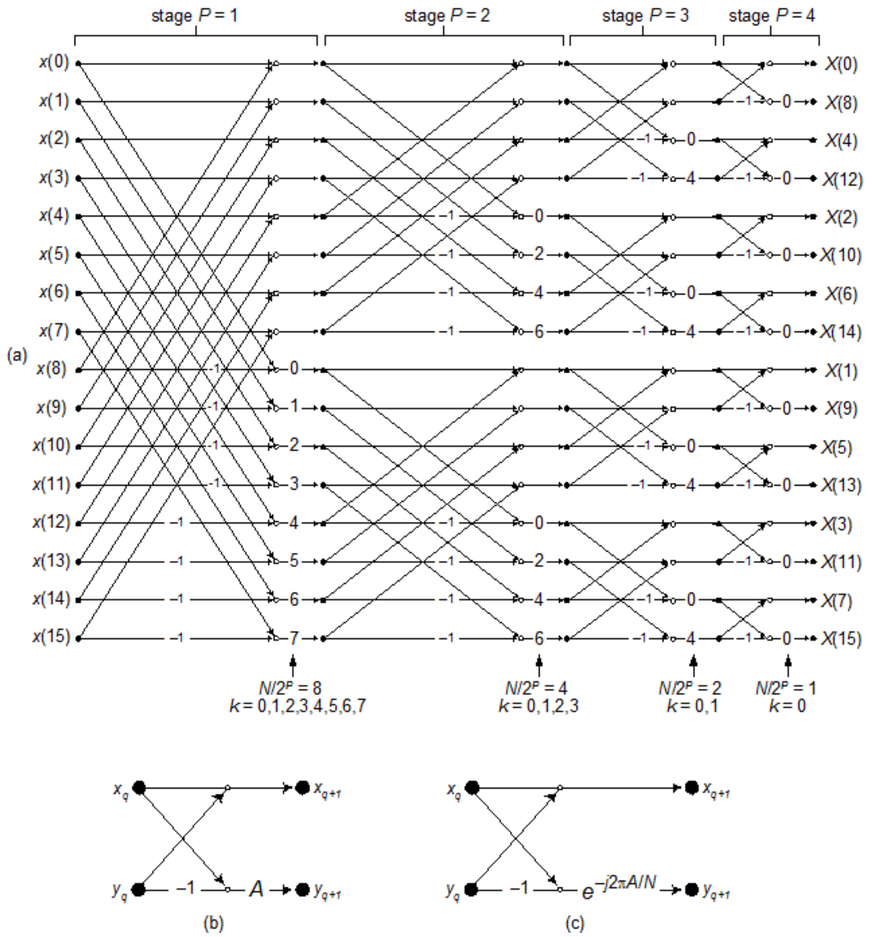
- Useful for editing!

- We will use later for:
  - Denoising
  - Time stretching/ compression
  - Spectral manipulations
  - Fast convolutions
  - And many more …

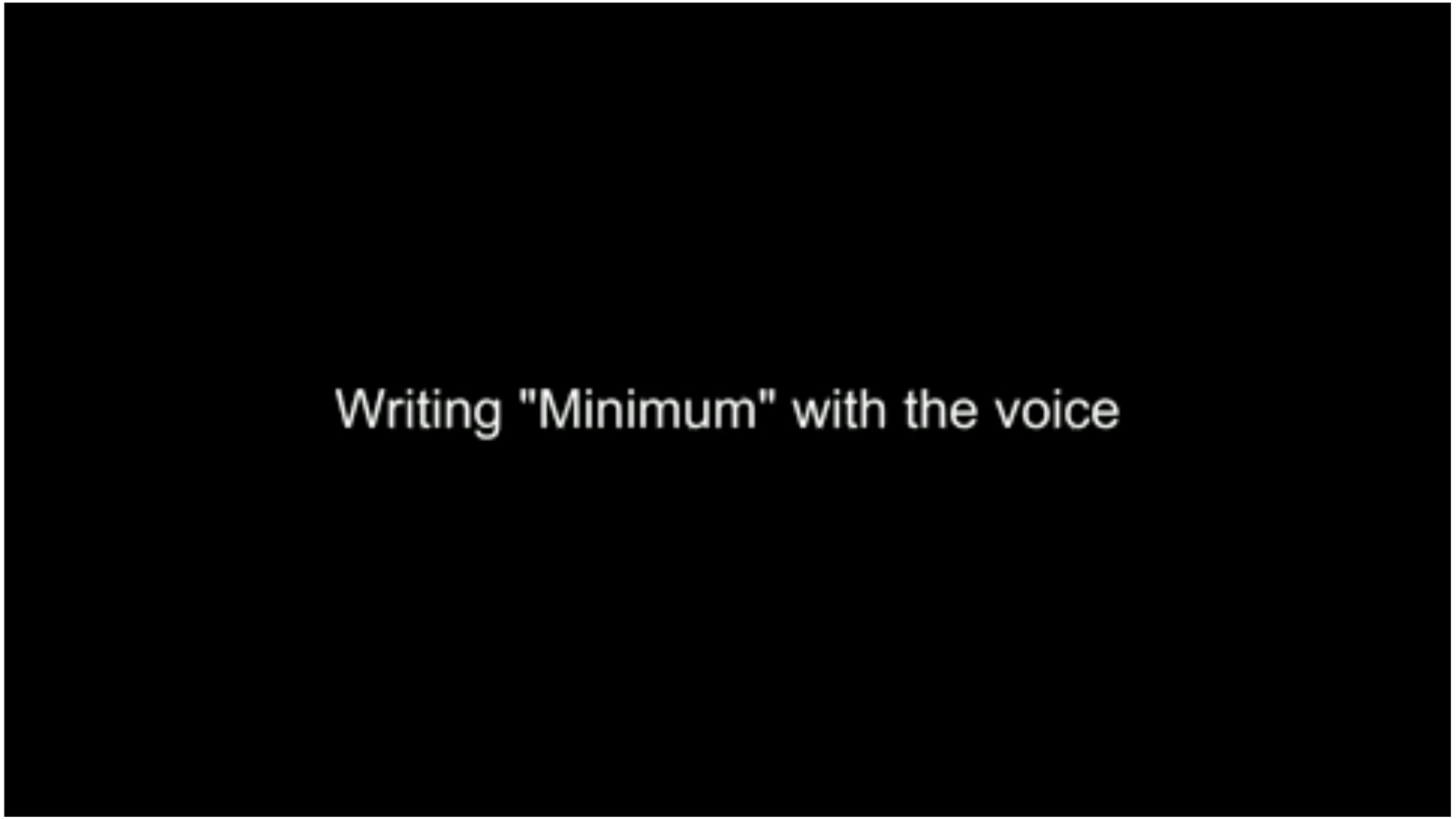


The Fast Fourier Transform (FFT)

- Efficient DFT algorithm
  - Huge speedup! (always use it!)
- Most routines you will find and use will be FFT routines
  - These return full complex spectrum
    - Some are specifically for real inputs
  - You might have to modify for real inputs



Minimum!!



https://www.youtube.com/watch?v=faBFiEfPxUU

Recap

- Digitizing and discretizing audio
  - Basic things to remember to represent sound best
- Frequency analysis and the DFT
- Time-frequency analysis and the spectrogram
  - Also its inverse

Reference material

- Overview of DSP:
  - http://www.dspguide.com/pdfbook.htm

- Spectral analysis of audio:
  - http://www.dsprelated.com/dspbooks/sasp/

Friday is lab day again

- Upcoming lab task
  - Implementing a forward/inverse spectrogram
  - Examining sounds using your code

- Will release assignment later today